Mathematical Foundations and Design Specification for Community Simulation Engines

From Conway to Dwarf Fortress to La Vida Misma

Project Vida Misma

2026-05-29

Contents

List of Figures

List of Tables

1 Cellular Automata

This section establishes the mathematical foundations of emergence: the phenomenon whereby simple, local transition rules applied uniformly across a spatial lattice produce complex, often unpredictable global behavior. Cellular automata (CAs) are the minimal formal system in which this phenomenon can be studied rigorously, and they underpin most of the simulation components discussed in subsequent sections.

1.1 Formal Definition

A cellular automaton is defined as a tuple $A = (L, S, N, f)$ where:

- L is the *lattice*: a regular grid, typically $\mathbb{Z}^d$ for $d \in \{1, 2, 3\}$.
- S is a finite set of states. The minimal binary case is $S = \{0, 1\}$.
- $N \subset L$ is the *neighborhood*: a finite set of relative cell positions that influence each cell's update.
- $f : S^{|N|} \to S$ is the *local transition function*, applied synchronously to every cell at each discrete time step t.

The global configuration $C_t : L \to S$ maps each cell to its state at time t. The global dynamics are fully determined by iterating f across all cells in parallel. No cell has access to global information; each cell's next state depends only on its current neighborhood.

The lattice geometry and neighborhood structure determine the class of patterns that can emerge. The standard neighborhoods are:

Name	Cardinality	Definition	Typical application
Von Neumann	$2d + 1$	$\{c : \|c\|_1 \leq 1\}$	Diffusion processes
Moore	3^d	$\{c : \|c\|_\infty \leq 1\}$	Game of Life, fluid simulation

Extended ($r = 2$)	5^d	$\{c : \|c\|_\infty \leq 2\}$	Long-range pattern formation

The choice of neighborhood is not merely parametric: it defines the maximum speed of information propagation and the class of local interactions available to the system.

1.2 Historical Context: Self-Reproduction and Universality

Von Neumann introduced the cellular automaton framework in the late 1940s as a formal setting for the question of kinematic self-reproduction. His construction used 29 states per cell and demonstrated that a configuration could contain a description of itself and use that description to build a copy (Burks 1970). The result was primarily existential: it established that self-reproduction is possible within a deterministic, discrete framework, but the construction was not intended for practical simulation.

Subsequent simplifications by Ulam, Burks, and others reduced the state space while preserving the core property: that local synchronous update rules can produce global configurations with non-trivial structure. The question of *how simple* such a system can be while still exhibiting complex behavior motivates the next development.

1.3 Conway's Game of Life (1970)

John Horton Conway's Game of Life (GoL) is a binary-state CA on $\mathbb{Z}^2$ with Moore neighborhood, governed by the rule B3/S23. For a cell with state s and neighbor sum n (the number of live cells among its 8 neighbors):

$$f(s, n) = \begin{cases} 1 & \text{if } s = 0 \text{ and } n = 3 \\ 1 & \text{if } s = 1 \text{ and } n \in \{2, 3\} \\ 0 & \text{otherwise} \end{cases}$$

The rule was selected by exhaustive experimentation to produce dynamics in between trivial extinction and unbounded growth. The specific numerical thresholds (birth at exactly 3, survival at 2 or 3) define a narrow operating regime where both stability and change are possible.

GoL exhibits several mathematically significant properties:

- **Turing completeness**: Rendell (2011) constructed a universal Turing machine within GoL, and subsequent work demonstrated arbitrary computation including a Tetris implementation. This means GoL can simulate any algorithm computable by a Turing machine.

- **Undecidability of the halting problem**: There is no general algorithm that, given an arbitrary initial configuration, can determine whether it will eventually reach a stable state. This follows directly from Turing completeness.
- **Maximum propagation speed**: Information cannot travel faster than 1 cell per tick, denoted c by analogy with the speed of light in physics. This constrains the maximum velocity of any self-propagating structure.

The persistent patterns that arise in GoL are classified by their temporal behavior:

Pattern type	Definition	Examples
Still life	$C_{t+1} = C_t$	Block, Beehive, Loaf
Oscillator	$C_{t+k} = C_t$ for minimal $k > 1$	Blinker ($k = 2$), Pulsar ($k = 3$)
Spaceship	$\exists \vec{v} \neq \vec{0} : C_{t+k} = C_t + \vec{v}$	Glider ($c/4$ diagonal), LWSS ($c/2$ orthogonal)
Gun	Produces spaceships periodically	Gosper Glider Gun (period 30)

The **Hashlife** algorithm (Gosper, 1980s) exploits the hierarchical structure of quadtrees to memoize sub-pattern evolution. By caching the outcome of spatial regions at multiple time scales, it achieves exponential acceleration for sparse, regular patterns. In benchmark settings it has computed configurations to generation 6.3×10^{24} in seconds, though its performance degrades for chaotic, non-repeating configurations.

GoL serves as the foundational example for this document: it demonstrates that a transition function with only two states and a 3x3 neighborhood can produce dynamics that are formally undecidable. The implication for simulation design is that computational intractability at the macro level does not require complexity at the micro level.

1.4 Wolfram's Classification (1984)

Stephen Wolfram performed a systematic survey of one-dimensional CAs in the 1980s and observed that their long-term behavior falls into four qualitative classes (Wolfram 1984):

Class	Dynamics	Dynamical systems analogue
I	Convergence to a homogeneous fixed point	Point attractor

Class	Dynamics	Dynamical systems analogue
II	Convergence to periodic structures	Limit cycle
III	Aperiodic, apparently random dynamics	Strange attractor / chaos
IV	Localized persistent structures that propagate and interact	Computation / complex transients

Class IV is of particular interest because it is the regime where structures analogous to GoL's gliders arise: localized patterns that maintain coherence over time, propagate through space, and interact non-trivially. Wolfram conjectured, and Cook later proved for Rule 110, that Class IV CAs can be Turing complete.

Langton formalized the boundary between these classes by introducing the λ parameter: the fraction of the rule table that maps to a non-quiescent state (Langton 1990). Low λ corresponds to Class I/II (ordered), high λ to Class III (chaotic), and intermediate λ to Class IV. This "edge of chaos" picture suggests that complex computation in CAs is a critical phenomenon, arising at a phase transition between order and disorder.

This classification has practical implications for simulation design: the behavior of an emergent system is highly sensitive to the parameter regime. Rules that are too simple produce stasis; rules that are too complex produce noise. Calibrating the rule set to operate near the critical boundary is essential for generating interesting dynamics.

1.5 Application: Fluids in Dwarf Fortress

Dwarf Fortress (Adams, 2002–present) implements water and magma dynamics as a three-dimensional CA. Each tile contains a fluid level $s \in \{0, 1, \ldots, 7\}$. The update rules are:

1. **Gravity**: if the tile below has $s_{\text{below}} < 7$, transfer $\min(s_{\text{current}}, 7 - s_{\text{below}})$ units downward.
2. **Lateral spread**: if the tile below is full ($s = 7$) and a lateral neighbor has $s_{\text{neighbor}} < s_{\text{current}}$, transfer fluid laterally.
3. **Pressure**: resolved via flood-fill from pressure sources, allowing fluid to propagate upward through enclosed channels.

This is an approximation of fluid dynamics that deliberately sacrifices physical accuracy for computational efficiency and player legibility. The 8-level quantization was chosen so that players can visually distinguish fluid levels at a glance. The system does not model viscosity, turbulence, or surface tension, yet it produces behavior that players perceive as fluid-like: flow, accumulation, flooding, and pressure-driven eruption.

The fluid system illustrates a design principle that recurs throughout this document: the gap between a physically accurate model and a *behaviorally adequate* one can be exploited to reduce computational cost without sacrificing the qualitative phenomena that drive emergent narrative.

1.6 From Discrete to Continuous: SmoothLife and Lenia

The binary state space and discrete time of classical CAs are design choices, not mathematical necessities. Two generalizations relax these constraints.

SmoothLife, introduced by Rafler (2011), replaces the binary cell state with a continuous value $f(\vec{x}, t) \in [0, 1]$ and the neighbor count with continuous integrals over annular regions:

$$m(\vec{x}, t) = \frac{1}{M} \int_{|\vec{u}|<r_i} f(\vec{x} + \vec{u}, t)\, d\vec{u} \tag{1}$$

$$n(\vec{x}, t) = \frac{1}{N} \int_{r_i<|\vec{u}|<r_a} f(\vec{x} + \vec{u}, t)\, d\vec{u} \tag{2}$$

where m is the inner filling (cell interior) and n is the outer filling (annular neighborhood). The hard thresholds of GoL are replaced by smooth sigmoid functions:

$$\sigma_1(x, a) = \frac{1}{1 + \exp\left(-\frac{4(x-a)}{\alpha}\right)} \tag{3}$$

$$s(n, m) = \sigma_2\left(n, \sigma_m(b_1, d_1, m), \sigma_m(b_2, d_2, m)\right) \tag{4}$$

and the discrete update becomes a continuous time evolution:

$$\partial_t f(\vec{x}, t) = S[s(n, m)] \cdot f(\vec{x}, t) \tag{5}$$

SmoothLife produces translating structures ("smooth gliders") that can propagate in any direction, not just the 8 directions of a Moore neighborhood. The significance of this result is that it demonstrates emergence does not depend on spatial discretization: continuous dynamics can produce qualitatively similar phenomena.

Lenia, introduced by Chan (2019), generalizes further by introducing learnable growth functions and parameterized kernels:

$$\frac{dA}{dt} = G(K * A^t) \tag{6}$$

where A^t is the grid state, G is a growth function (typically a Gaussian-shaped function centered at zero), K is a kernel (typically a ring-shaped radial function), and $*$ denotes 2D convolution. By varying G and K across a parameter space, Chan identified over 400 distinct self-organizing patterns ("species") with qualitatively different behaviors: stable orbits, reproduction, predation, and collective motion. Chan's classification of these patterns into families and genera parallels biological taxonomy.

However, Davis (2022) demonstrated that the self-organization in Lenia is critically dependent on numerical precision. The glider *Scutium gravidus*, for example, destabilizes when simulation precision is increased from float32 to float64, when spatial resolution is increased via larger kernels, or when temporal resolution is increased via smaller time steps. This suggests that the "species" discovered in Lenia are not properties of the continuous mathematical system alone, but co-products of the specific numerical approximation used. Davis frames this as a form of numerical embodied cognition: the patterns that emerge are adapted to the computational substrate on which they were discovered.

The practical implication for simulation engine design is that numerical representation choices (float precision, temporal step size, spatial resolution) are not merely performance parameters. They can qualitatively affect which behaviors emerge. This is particularly relevant when choosing data representations for large-scale community simulations.

1.7 Neural Cellular Automata

Sandler et al. (2020) replaced the hand-designed transition function with a learned one:

$$s_{t+1} \leftarrow S(s_t, i; \theta) \tag{7}$$

where S is a 3-layer convolutional neural network with residual connections ($S(s) = U(s) + s$) parameterized by θ, and i is a persistent external input (the image to segment). The model was trained via mini-unroll cross-entropy loss to perform image segmentation using fewer than 10,000 parameters.

Several findings from this work are relevant to simulation design. First, the recurrent application of a purely local rule can solve tasks that appear to require global information (e.g., distinguishing object from background across the entire image). Second, the specific spatial filters learned were of modest importance: replacing them with random filters degraded but did not destroy performance, suggesting that the recurrent dynamics of the CA loop itself is the primary source of computational power. Third, the training process exhibited sharp transitions ("regime changes") from unstable to stable dynamics, qualitatively similar to phase transitions in physical systems.

This work establishes a connection between CA theory and deep learning that is

relevant to simulation engine design: it raises the possibility that agent interaction rules could be learned from data rather than hand-coded, while preserving the local, synchronous, emergent character of CA dynamics.

2 Procedural Terrain Generation

Cellular automata provide the formal framework for *dynamics*—how the world evolves over time. Procedural generation addresses a distinct problem: constructing the initial *topology* of the simulation world. This section covers the mathematical techniques used to produce terrain, biomes, and spatial structures that exhibit statistical properties similar to natural landscapes.

2.1 Gradient Noise: Perlin Noise (1983)

Ken Perlin developed gradient noise while working on computer-generated textures for the film *Tron* (1982). The problem was that existing methods produced either constant-valued regions or uncorrelated white noise. Neither captured the statistical structure of natural phenomena like clouds, terrain, or organic textures, which exhibit correlated variation across multiple spatial scales.

Perlin's method assigns a random unit-length gradient vector $\vec{g}_i$ to each node of a regular grid. For any query point $\vec{P}$ within a grid cell, the algorithm computes dot products between the gradient at each corner and the displacement vector:

$$\text{dot}_i = \vec{g}_i \cdot (\vec{P} - \vec{c}_i) \tag{8}$$

where $\vec{c}_i$ is the position of corner i. The 2^d dot products (for dimension d) are then interpolated using a smoothing function. The original formulation used a cubic Hermite polynomial:

$$f(x) = 3x^2 - 2x^3 \tag{9}$$

This function satisfies $f(0) = 0$, $f(1) = 1$, and $f'(0) = f'(1) = 0$, but its second derivative is discontinuous at cell boundaries, producing visible artifacts. The improved version uses a quintic polynomial that is C^2 continuous:

$$f(x) = 6x^5 - 15x^4 + 10x^3 \tag{10}$$

satisfying $f'(0) = f'(1) = 0$ and $f''(0) = f''(1) = 0$.

The output is a continuous scalar field in $[-1, 1]$ with controlled spectral properties. To produce the multi-scale variation characteristic of natural terrain, multiple *octaves* are superimposed:

$$\text{noise}_{\text{total}}(\vec{x}) = \sum_{i=0}^{O-1} \text{perlin}(\vec{x} \cdot f_0 \cdot l^i) \cdot a_0 \cdot p^i \tag{11}$$

where O is the number of octaves, l is the *lacunarity* (frequency multiplier between successive octaves, typically $l \approx 2$), and p is the *persistence* (amplitude decay, typically $p \approx 0.5$). This technique, known as *fractal Brownian motion* (fBm), produces a power spectrum that approximates $1/f^\beta$ noise—a statistical signature common in natural terrain, coastlines, and cloud formations.

Parameter	Effect on output	Typical range
Base frequency f_0	Scale of the largest features	0.001–0.1 (relative to map size)
Octaves O	Amount of high-frequency detail	4–12
Persistence p	Contribution of each successive octave	0.4–0.7
Lacunarity l	Frequency ratio between octaves	1.8–2.2

Perlin received a Technical Academy Award in 1997 for this work. **Simplex noise** (Perlin, 2001) improved on the original by replacing the hypercubic grid with a simplex lattice, reducing computational complexity from $O(2^d)$ to $O(d)$ in dimension d and eliminating directional artifacts inherent to the axis-aligned grid structure.

2.2 Diamond-Square: Midpoint Displacement on a Grid

An alternative to gradient noise for terrain generation is the *diamond-square* algorithm, a specific instance of midpoint displacement fractals. The method was motivated by Mandelbrot's observation (1967) that natural forms such as coastlines and mountain ridges exhibit statistical self-similarity: their structure recurs at multiple spatial scales.

The algorithm operates on a $2^n + 1 \times 2^n + 1$ grid:

1. Initialize the four corners with random values.
2. **Diamond step**: set the center of each square to the average of its four corners plus a random displacement $\delta \sim \mathcal{U}(-d_n, d_n)$.
3. **Square step**: set the midpoint of each edge to the average of its available neighbors plus displacement δ.
4. Reduce the displacement magnitude:

$$d_{n+1} = d_n \times 2^{-H} \tag{12}$$

5. Repeat on each sub-square until the grid is fully populated.

The parameter H (roughness) controls the fractal dimension of the output. Higher H produces smoother terrain; lower H produces more rugged terrain. The relationship to fractal dimension is $D = 3 - H$ for a 2D height field.

Dwarf Fortress uses diamond-square for elevation maps rather than Perlin noise, reportedly for reasons of implementation simplicity and predictability of output on a regular grid.

2.3 Wave Function Collapse: Constraint Satisfaction for Tile Maps (2016)

Gradient noise and midpoint displacement generate continuous scalar fields. A different problem arises when the goal is to generate *discrete, coherent structures*: tile maps where adjacent tiles must satisfy compatibility constraints (e.g., "a door tile requires wall tiles on both sides"), or patterns that must match a given exemplar image.

Wave Function Collapse (WFC) (Gumin 2017) formulates this as a constraint satisfaction problem. Each cell c_i maintains a set of allowed states $S_i \subseteq \Sigma$ (the "superposition"). The algorithm iterates:

1. **Observe**: select the cell c_i with minimum Shannon entropy among uncollapsed cells:

$$H(c_i) = -\sum_{s \in S_i} p_s \log_2 p_s \tag{13}$$

where p_s is the prior probability of state s.

2. **Collapse**: set $S_i = \{s\}$ for a randomly chosen s, weighted by prior probabilities.

3. **Propagate**: enforce arc consistency—for each neighbor c_j of a recently collapsed cell, remove states from S_j that are incompatible with the collapsed state. Continue propagation until no further eliminations occur.

4. If a cell's allowed set becomes empty (contradiction), backtrack or restart.

The minimum-entropy heuristic (step 1) is a standard variable-ordering strategy from the CSP literature, analogous to the "most constrained variable" heuristic. Karth and Smith (2017) formalized this connection, demonstrating that WFC is arc consistency checking with a specific variable-ordering heuristic.

WFC operates in two modes: *tile-based* (adjacency constraints specified explicitly) and *overlapping* (constraints extracted automatically from an exemplar image by analyzing all $N \times N$ sub-patterns). The overlapping mode has been widely adopted in procedural generation for games including *Caves of Qud*, *Townscaper*, and *Bad North*.

2.4 Layered Generation: The Dwarf Fortress Pipeline

Dwarf Fortress generates worlds through a sequential pipeline where each layer depends on previously computed layers. Adams describes this as an instance of his second design principle: decompose the system into independent subsystems and let complex phenomena emerge from their interaction (Adams 2014).

Layer	Output	Primary method
Elevation	Height map (0–400)	Diamond-square fractal
Temperature	Per-tile temperature	Function of elevation + latitude + noise
Rainfall	Per-tile precipitation	Noise + orographic shadow model
Drainage	Per-tile drainage value (0–100)	Separate fractal
Vegetation	Per-tile vegetation density	Derived from rainfall + temperature + drainage
Biome classification	Per-tile biome type	Threshold intersection of all previous layers
Rivers	River paths	Gradient descent on elevation with drainage constraints
Civilizations	Settlements and territories	Voronoi-based territory assignment with historical simulation

The key property of this pipeline is that each layer introduces a small number of independent parameters, but their *intersection* produces rich combinatorial variation. A "desert" is not coded as a single entity; it emerges where elevation, temperature, rainfall, and drainage simultaneously satisfy the desert criterion. This produces geographic features that are internally consistent in ways that a monolithic biome generator would not guarantee.

2.5 Diamond-Square vs. Perlin Noise: A Comparison

Property	Perlin/Simplex noise	Diamond-square
Grid requirement	Arbitrary resolution	Must be $2^n + 1$
Continuity	C^2 (quintic) or C^1 (cubic)	Discontinuous at boundaries
Directional artifacts	Axis-aligned (Perlin); isotropic (Simplex)	None inherent
Self-similarity	Via fBm octave stacking	Built into the algorithm

Property	Perlin/Simplex noise	Diamond-square
Implementation complexity	Moderate	Low
Extensibility to $d > 2$	Straightforward	Requires generalization
Control over spectrum	Explicit (octaves, persistence)	Indirect (roughness parameter)

3 Agent-Based Modeling and Artificial Intelligence

This section separates general agent-based modeling theory from the mechanisms implemented in *La Vida Misma*. A statement marked **Background** motivates the model but does not describe executable behavior. **Implemented** identifies a current mechanism, **Design objective** identifies a criterion used to evaluate it, and **Not established** identifies a hypothesis for which the present experiments do not provide sufficient evidence.

3.1 Agent-Based Modeling

Background. Agent-based modeling (ABM) studies systems in which autonomous, heterogeneous entities interact with one another and with an environment. The ODD protocol organizes a model description into Overview, Design concepts, and Details, and asks the author to state explicitly what agents sense, how they adapt, where stochasticity enters, and which aggregate observations are measured (Grimm et al. 2020). This is especially useful for distinguishing a programmed micro-mechanism from an observed macro-pattern.

Classic models such as Sugarscape show how resource distributions, bounded vision, metabolism, and local movement can generate aggregate distributions that are not stored as agent goals (Epstein and Axtell 1996). Such results are theoretical precedents, not evidence that the same phenomena occur in this simulation.

Implemented. An inhabitant stores individual needs, personality, stress, skills, inventory, spatial memory, lifecycle state, and a private random stream. Systems update those components in a fixed tick pipeline. No action is assigned by profession, community label, or `agent.id`; graph communities and founder archetypes are observational or initialization data rather than commands.[1]

[1] Implementation references: `src/components.h`, `src/simulation.cpp`, and `src/sim_lifecycle.cpp`. The identity-routing and group-label boundaries are audited by `tests/verify_policy_audit.cmake`.

3.2 Feasible Utility and Stochastic Choice

Background. Utility AI assigns comparable scores to heterogeneous actions. A deterministic implementation would choose an argmax. A Boltzmann policy instead turns scores into probabilities and approaches greedy choice as its temperature approaches zero.

The executable retains that deterministic limit when the configured temperature is effectively zero:

$$a_t^* = \arg\max_{a \in C_t} U_t(a). \tag{14}$$

This is a fallback, not the canonical policy at temperature 0.4.

Implemented. At a decision tick, the simulation computes a score and a feasibility flag for each of thirteen actions. An action with no feasible target or effect receives no selection weight, and a score of zero also receives no weight. `IDLE` is always feasible and has the explicit positive score 0.02. The recorded diagnostic decomposition is

$$U_t(a) = U_{\text{self},t}(a) + U_{\text{factory},t}(a) - U_{\text{cost},t}(a) - U_{\text{risk},t}(a). \tag{15}$$

At present, each action's fully shaped score is classified as either `self` or `factory`, and explicit cost and risk remain zero; the equation describes the implemented metrics contract, not four independently calibrated score models. For the resulting candidate set

$$C_t = \{a : \text{feasible}_t(a) \wedge U_t(a) > 0\},$$

selection uses

$$\Pr(A_t = a) = \frac{\exp((U_t(a) - U_{\max})/\tau)}{\sum_{b \in C_t} \exp((U_t(b) - U_{\max})/\tau)}, \qquad a \in C_t, \tag{16}$$

where $U_{\max} = \max_{b \in C_t} U_t(b)$ is a numerical-stability offset and the canonical temperature is `selection_temperature = 0.4`. If the temperature is effectively zero, the code uses a greedy argmax. Action commitment persists for several ticks, subject to feasibility and survival overrides, so choice is not an independent draw on every tick.[2]

The score is not the simple quadratic expression used in earlier drafts. The canonical configuration selects urgency curve variant 3 for hunger and rest,

[2]Implementation references: `src/sim_utility.cpp` and `src/sim_targets.cpp`. `test_metrics_contract` and `test_build_can_be_disabled` in `tests/simulation_tests.cpp` check explicit feasible `IDLE`, zero target failures, and zero softmax weight for utility-zero actions.

continuous stress variant 1, personality-dependent response thresholds, mood, skill, local supply observations, social learning, and action-specific gates. Consequently, a compact universal equation for every action would be misleading; the executable definitions are in `src/sim_utility.cpp` and the effective tuning is in `config/default.toml`.

For social, expression, and purpose, the implemented higher-need transform is

$$u_{\text{higher}}(n) = n^{\alpha}, \qquad \alpha = 2 \tag{17}$$

under the canonical configuration. Hunger and rest do **not** use this equation: canonical urgency variant 3 uses the scaled logistic $8/(1 + e^{-12(n-0.7)})$.

3.3 Local Information and Technical Pathfinding

Design objective. Decisions should be explainable from individual state, observations, and bounded memory rather than colony-wide omniscience.

Implemented, with a limitation. Most utility and target queries inspect a Manhattan radius of twelve tiles (`Simulation::OBSERVATION_RADIUS`). Food, materials, machines, storage, people, artifacts, and candidate places outside that radius do not enter ordinary choice. A remembered place may be revisited, but its current remote conditions are not read. Once a visible or remembered target has been chosen, cached A* may inspect the full grid as a technical routing service; that service does not choose the behavioral goal.

Conveyor construction is not fully local. `Grid::find_conveyor_build_site()` scans factory-wide machine, belt, storage, and Exit connectivity and runs a grid-wide breadth-first route planner before callers reject a returned build site farther than radius twelve. Parts of conveyor urgency also count factory-wide machine connections. Thus the defensible claim is **mostly local decision-making**, not complete epistemic locality. The counterfactual test changes distant food stock and verifies identical utility, action, and target, but there is not yet an equivalent proof for conveyor planning.[3]

3.4 The Relationship Graph

Implemented. Relationships form a directed graph over stable historical agent IDs. Each ordered edge stores two distinct variables:

$$R_{ij} = (f_{ij}, t_{ij}) \in [0, 1] \times [-1, 1], \tag{18}$$

- `familiarity` in $[0, 1]$, meaning accumulated evidence of contact;

[3] Implementation references: `src/simulation.h`, `src/sim_utility.cpp`, `src/sim_targets.cpp`, and `src/grid.h`. The distant stock counterfactual is `test_unseen_stock_does_not_change_decision`; planner purity is `test_conveyor_planning_is_pure` in `tests/simulation_tests.cpp`.

- `trust` in $[-1, 1]$, meaning the observer's evaluation of the other person.

The distinction is causal. Copresence within two tiles increases familiarity in both directions without creating trust. Effective shared `BUILD`, `WORK`, or `CREATE` activity adds reciprocal familiarity and trust. `SOCIALIZE` is a stronger reciprocal positive interaction. Help is directional: the recipient gains trust in the helper, while the helper gains familiarity but not trust by fiat. A negative observation modifies only `observer -> actor`. Familiarity decays after inactivity and trust drifts toward neutral.[4]

Trust and familiarity affect social target scores, food sharing, collaboration, grief, opinion exchange, stress contagion, and an influence score. Every fifty ticks, reciprocal familiarity and trust can be used to derive connected graph components of at least three inhabitants. The resulting `community_id` is used for observation and metrics only; static policy tests prohibit utility, targeting, execution, and institutional policy from consuming it.

The stress system is likewise stateful but does not implement the three memory layers described in older drafts. At the dedicated stress-update stage its scalar update can be written exactly as

$$\sigma' = \max(0, \min(1, \sigma + I_t) - d_0[1 + 8(1 - h)(1 - r)]), \qquad d_0 = 0.005, \quad (19)$$

where I_t is the resilience-modulated input from critical hunger, critical rest, disease, and, only in legacy policy variant 0, noncompliance. Action effects, trauma, and later graph contagion are separate stages; section 14.7 describes them.

3.5 Schelling as Background, Not a Model Claim

Background. Schelling's segregation model assigns agents a same/different neighborhood preference and a tolerance threshold; relocation under that rule can produce segregation without a central segregating authority (Schelling 1971).

Not implemented. *La Vida Misma* has no same/different class, no Schelling tolerance threshold, and no rule that moves an inhabitant because neighbors are dissimilar. Place choice instead scores remembered outcomes and currently observable properties such as traffic, machinery noise, hazard, food access, known people, artifacts, and travel distance. It is therefore not formally a Schelling process.

Not established. The current paired experiments do not establish spatial segregation, subcultures, leadership, or free-riding. Those terms require a pre-

[4] Implementation reference: `src/social.h` and the social and community systems in `src/simulation.cpp`. Directionality and label neutrality are covered by `test_social_evidence_is_directional` and `test_graph_labels_are_behavior_neutral`.

declared metric, a relevant disabled-mechanism or shuffled counterfactual, multiple seeds, and uncertainty over a sufficiently long horizon. The measured place-affinity result is narrower and is reported in section 15.

4 Pathfinding

Agents must navigate the spatial environment to satisfy their needs. Pathfinding algorithms compute sequences of positions that connect an agent's current location to a goal while respecting the traversability constraints of the terrain. This section covers the standard approaches and their computational trade-offs.

4.1 A*: Informed Search on a Graph

A* (Hart et al. 1968) is the standard pathfinding algorithm for grid-based worlds. It extends Dijkstra's algorithm with an admissible heuristic $h(n)$ that estimates the cost from node n to the goal:

$$f(n) = g(n) + h(n) \tag{20}$$

where $g(n)$ is the accumulated cost from the start node and $h(n)$ is the heuristic estimate. A* is guaranteed to find the optimal path when h is admissible (never overestimates the true cost) and consistent ($h(n) \leq c(n, n') + h(n')$ for all edges (n, n')).

Common heuristics for grid worlds:

Heuristic	Formula	Properties
Manhattan	$\|x_1 - x_2\| + \|y_1 - y_2\|$	Admissible for 4-connected grids
Euclidean	$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$	Admissible for any connectivity
Chebyshev	$\max(\|x_1 - x_2\|, \|y_1 - y_2\|)$	Admissible for 8-connected grids
Octile	$\max(dx, dy) + (\sqrt{2} - 1) \min(dx, dy)$	Admissible for 8-connected grids with uniform cost

A* has worst-case complexity $O(b^d)$ where b is the branching factor and d is the solution depth. In practice, the heuristic prunes the search space significantly, but large maps or many simultaneous agents can create computational bottlenecks.

4.2 Jump Point Search

Harabor and Grastien (2012) observed that on uniform-cost grids, many nodes expanded by A* are structurally irrelevant: they lie on straight corridors or open areas where the optimal path cannot deviate. Jump Point Search (JPS) exploits this by "jumping" over such nodes, expanding only *jump points*—nodes where the path is forced to turn.

JPS expands up to 100x fewer nodes than A* on uniform-cost grids while returning identical paths. The limitation is that it applies only when all traversable tiles have equal movement cost. For variable-cost terrain (swamps that slow movement, roads that accelerate it), A* with appropriate edge weights remains necessary.

4.3 Connected Components and Path Caching

Two practical optimizations reduce the cost of pathfinding in community simulations:

Connected components: Precompute which tiles belong to the same connected region (via flood-fill on the traversability graph). When an agent requests a path, first check whether the start and goal share a component. If not, the path is impossible and the search can be aborted immediately. This avoids the cost of a full A* expansion for unreachable goals.

Path caching: Maintain an LRU cache of recently computed paths. When the map changes (a wall is constructed, a flood occurs), invalidate only the paths that traverse the affected region. Path caching exploits the temporal locality of agent behavior: agents in the same area tend to request similar routes.

These optimizations are not asymptotically interesting but are critical for simulation performance. A community of 50+ agents requesting paths every few ticks on a large map will spend a disproportionate fraction of compute time in pathfinding unless these optimizations are applied.

4.4 Hierarchical Pathfinding

For very large maps, hierarchical approaches decompose the world into regions (e.g., rooms, districts) and compute paths in two phases: first at the region level (abstract graph), then within each region (concrete graph). This reduces the effective search space from the full map to a small number of intra-region searches. The trade-off is that hierarchical paths may be slightly suboptimal at region boundaries, but this is typically negligible for community simulation purposes.

5 Physical Simulation Systems

Physical systems in a community simulation must be computationally inexpensive enough to run alongside agent AI, pathfinding, and social simulation, while producing behavior that is qualitatively plausible. This section covers three systems that model physical phenomena using discrete approximations rather than continuous equations: fluid dynamics, heat transfer, and structural mechanics.

5.1 Fluid Dynamics as a Cellular Automaton

The Navier-Stokes equations provide the physically accurate model for fluid behavior, but their computational cost (requiring iterative solvers on fine meshes at small time steps) makes them impractical for real-time simulation with many concurrent systems. The cellular automaton approach, as used in Dwarf Fortress, approximates fluid behavior with a quantized, rule-based system.

Each tile contains a fluid level $s \in \{0, 1, \ldots, 7\}$. The update rules are applied in order:

1. **Gravity**:

$$s_{\text{below}} < 7 \implies \text{transfer} = \min(s_{\text{current}}, 7 - s_{\text{below}}) \tag{21}$$

2. **Lateral spread** (when the tile below is full):

$$s_{\text{neighbor}} < s_{\text{current}} \implies \text{transfer} = \left\lfloor \frac{s_{\text{current}} - s_{\text{neighbor}}}{2} \right\rfloor \tag{22}$$

3. **Pressure propagation**: flood-fill from pressure sources to identify connected fluid bodies, then equalize levels across connected tiles, allowing fluid to rise above its entry point in enclosed channels.

This system does not conserve momentum, does not model viscosity or surface tension, and does not produce turbulence. However, it reproduces several qualitatively important behaviors: downward flow under gravity, lateral spreading on flat surfaces, accumulation in basins, and pressure-driven rising in enclosed columns. These are sufficient for gameplay scenarios (flooding, irrigation, magma intrusion) at a computational cost of $O(\text{fluid tiles})$ per tick.

5.2 Heat Diffusion

Heat transfer is modeled as discrete diffusion on the grid. Each tile has a temperature T and a material-specific conductivity k. The update rule is:

$$T_t^{i,j} = T_{t-1}^{i,j} + \alpha \cdot k_{i,j} \cdot \sum_{(n,m) \in N(i,j)} (T_{t-1}^{n,m} - T_{t-1}^{i,j}) \tag{23}$$

where α is a global scaling constant and $N(i, j)$ is the neighborhood of tile (i, j). This is a discrete approximation of the continuous heat equation:

$$\frac{\partial T}{\partial t} = k\nabla^2 T$$

The discrete update converges to the continuous solution as the grid resolution increases and the time step decreases. For simulation purposes, the discrete version is sufficient: it produces the expected qualitative behavior—heat flows from hot to cold regions, materials with higher conductivity equilibrate faster, and insulated regions retain heat longer.

The stability condition for the explicit Euler scheme is:

$$\alpha \cdot k_{\max} \cdot |N| < 1$$

Violating this condition produces numerical instability (temperature oscillations that grow without bound). In practice, this constrains the choice of α given the maximum conductivity in the simulation.

5.3 Structural Collapse

Structural mechanics in community simulations typically use a simplified connectivity model rather than a stress analysis. The rule is:

1. Each constructed tile (wall, floor, etc.) must be *connected to the ground* via a continuous path through adjacent constructed tiles.
2. Connectivity is checked via flood-fill from the ground layer.
3. Tiles that lose connectivity (due to mining, destruction, or collapse) enter a "falling" state.
4. Falling tiles propagate downward until they reach a supported surface or the ground.

This model captures the essential gameplay-relevant behavior—undermining a support causes the structure above to collapse—without computing stress vectors. It is computationally O(constructed tiles) when triggered by a modification event.

The connection between structural collapse and fluid dynamics is an example of emergent interaction between subsystems: a flood can destroy a support, triggering a collapse, which redirects the flood. Neither system references the other; the interaction occurs through the shared tile grid.

6 Social Simulation

Social simulation is the subsystem that distinguishes a community simulation from a terrain renderer with moving agents. It encompasses the economic system

(production, trade, specialization), the social graph (relationships, stress contagion), and the spatial embedding of social dynamics. This section describes how these components interact to produce emergent social phenomena.

6.1 The Production Economy

The economic system is modeled as a directed acyclic graph (DAG) of transformations:

$$\text{Inputs} \xrightarrow{\text{workshop + labor + time}} \text{Outputs}$$

Raw resources (wood, ore, stone) are transformed into intermediate goods (planks, bars, blocks) and then into final products (furniture, weapons, food). Each transformation requires an agent with sufficient skill, an appropriate workshop, and a time cost proportional to the agent's skill level.

The economy is *emergent* rather than scripted: prices are not set by a global controller but arise from the interaction of supply (what agents produce) and demand (what agents need). When the utility system evaluates tasks, it weights them by both urgency and distance:

$$U(\text{task}) = \text{urgency}(\text{task}) \times \frac{1}{1 + \beta \cdot d(\text{agent}, \text{task})} \tag{24}$$

where d is the path distance and β is a distance sensitivity parameter. This distance penalty produces natural industrial organization: agents prefer nearby tasks, which causes workshops to become associated with their input sources, forming de facto industrial districts.

This coupling between economic behavior and spatial layout is a design insight from Dwarf Fortress: in early versions, agents accepted any task regardless of distance, producing inefficient cross-fortress movement. Adams resolved this not by scripting "take the nearest task" but by making distance reduce utility in the decision system. The self-organization was not programmed; it emerged from a modification to the utility function.

6.2 Skill Progression and Specialization

Skills improve through practice. A typical progression model uses increasing XP thresholds:

$$\text{XP for level } N = \text{base} + \text{increment} \times N \tag{25}$$

Cumulative XP to reach level L:

$$\sum_{n=0}^{L-1}(\text{base} + \text{increment} \times n) = \text{base} \times L + \text{increment} \times \frac{L(L-1)}{2} \tag{26}$$

Skill level affects output quality. In Dwarf Fortress, quality tiers (base, well-crafted, finely-crafted, superior, exceptional, masterwork) have probability thresholds that depend on skill. A "Legendary" (level 15) craftsdwarf produces exceptional-quality items approximately 65% of the time, with a hard cap of 33.3% for masterwork.

This system creates a positive feedback loop: an agent who performs a task gains skill, which increases the quality of their output, which increases the utility of assigning them similar tasks (because higher-quality outputs are more valuable), which increases the probability they will be assigned those tasks again. The result is *natural specialization*: agents gravitate toward roles they have practiced, without any explicit assignment mechanism.

Skills can also decay through disuse, providing a countervailing force that prevents permanent lock-in and allows role flexibility in response to changing community needs.

6.3 Relationships and Social Dynamics

The social graph (defined in section 3.4) is the substrate on which social dynamics operate. The economic and skill systems feed into it through events: working alongside another agent modifies the relationship edge, successful collaboration strengthens it, competition weakens it, and traumatic events (injury, death, loss) create large negative modifications modulated by personality.

The stress system interacts with the social graph through contagion:

$$\Delta\text{stress}_j = \gamma \cdot |w(i,j)| \cdot \Delta\text{stress}_i \cdot \text{susceptibility}_j \tag{27}$$

where γ is a global contagion coefficient and $w(i,j)$ is the relationship weight. Positive relationships propagate stress (empathy response), while strongly negative relationships can also propagate stress (antagonism response). Agents with many strong relationships are both more supported and more vulnerable to cascading stress events.

This mechanism produces emergent social dynamics: a traumatic event (e.g., a creature attack) affects not only the direct victims but their entire social network, with intensity attenuated by graph distance. A community with dense, positive social connections recovers faster (shared coping) but is also more susceptible to cascading breakdown if a central agent is affected.

6.4 Spatial Game Theory

The social dynamics described above gain additional structure when agents are embedded in physical space. Nowak and May (1992) demonstrated that placing simple game-theoretic interactions on a spatial lattice fundamentally changes the evolutionary outcomes.

In the standard Prisoner's Dilemma, defection is the dominant strategy in a well-mixed population: agents who defect always outperform cooperators, leading to universal defection. However, when agents are placed on a 2D lattice and interact only with their spatial neighbors, cooperators can form clusters that resist invasion by defectors. The boundary of a cooperator cluster generates enough mutual benefit to offset the exploitation by surrounding defectors. The spatial structure itself enables cooperation that would be impossible without it.

This result has direct implications for community simulation design:

- The physical layout of a settlement (who lives next to whom, which workshops are adjacent, where resources are concentrated) is not merely aesthetic. It constrains which social dynamics can emerge.
- Cooperation, trade, and trust are more likely to develop between spatially proximate agents.
- Spatial isolation or segregation can produce divergent cultural or behavioral clusters within the same community.

A simulation engine that treats spatial position as a first-class variable in social interactions will produce richer emergent dynamics than one where social and spatial systems are independent.

6.5 RimWorld's Storyteller: A Meta-Agent for Pacing

RimWorld introduces an additional layer: a *Storyteller* meta-agent that calibrates the difficulty of external events (raids, weather, disease) to maintain narrative tension. The Storyteller does not simulate physical reality; it monitors the colony's state and adjusts the rate of adverse events:

$$P(\text{event at } t) = f(\text{wealth}_t, \text{population}_t, \text{time}_t) \tag{28}$$

where the function parameters differ between Storyteller personalities. The "Phoebe" storyteller uses a low baseline with long peaceful intervals; "Cassandra" uses increasing difficulty over time; "Randy" samples from a uniform distribution, producing unpredictable sequences.

The Storyteller is a meta-agent that operates at a different level of abstraction than the in-world agents. It does not follow the same rules; it observes the simulation state and injects events to modulate the emergent narrative. This architectural pattern—a separate monitoring system that adjusts parameters based

on observed state—is applicable to any community simulation where unmodulated emergence might produce extended periods of stagnation or catastrophic collapse.

7 Simulation Design Principles

The principles below are methodological constraints, not claims that every desired macro-phenomenon has emerged. They distinguish the architecture that exists, the objectives that guide it, and hypotheses that still require evidence.

7.1 Decomposition and Causal Legibility

Background. Decomposition replaces a scripted composite state with smaller processes that interact through shared state. This idea is common to simulation game design (Adams 2014) and to formal ABM descriptions such as ODD (Grimm et al. 2020). It increases the number of possible combinations, but complexity alone does not prove emergence.

Implemented. `Simulation::advance()` defines one ordered pipeline: resource regeneration and need decay; utility, targeting, movement, and execution; social and spatial learning; conveyor transport and shipping; institutional policy; artifacts, graph communities, stress, death, lifecycle, and metrics. Action scoring, target selection, movement, effects, conveyor transport, indifferent policy, lifecycle, and Director interventions are separate translation units.[5]

Design objective. Each aggregate explanation should be traceable through that pipeline. For example, reduced shipping updates external support, support changes future resource regeneration, and scarcity can later affect hunger and mortality. The explanation must not jump directly from "quota missed" to "agent punished."

7.2 Adequacy Rather Than Maximal Fidelity

Background. A model is adequate when it preserves distinctions relevant to its question at the observable scale; more physical detail is not automatically better.

Implemented. The executable uses a bounded 60×40 grid, five resource types, three machine recipes, directed single-content conveyors, and one-tick updates. It does not model continuous mechanics, fluid dynamics, or a third spatial dimension. Cached A* provides movement paths over the grid, while agent choice is mostly bounded to radius-twelve observations.

[5] `src/simulation.cpp` is the scheduling source; subsystem boundaries are declared in `src/simulation.h` and compiled through `VIDA_SIM_SOURCES` in `CMakeLists.txt`.

Design objective. Add fidelity only when it changes a measured decision, logistical constraint, or player-visible consequence. The current known exception to strict locality is conveyor planning, whose global route query should be treated as technical debt or explicitly reinterpreted as institutional planning, not silently described as local perception.

7.3 Hybrid ECS/Grid Architecture

Implemented. The runtime is hybrid rather than a pure Entity-Component-System:

Representation	Current responsibility
EnTT registry	inhabitants, artifacts, and their components
`Grid` dense arrays	tile type and `TileData` for terrain, resources, machines, storage, conveyors, zoning, and construction
Systems	transformations that read and write both representations
`SocialFabric`	dynamically sized directed trust/familiarity matrix keyed by stable agent ID
`Chronicle` and `SimulationMetrics`	factual event history and aggregate observation

An inhabitant is an ECS entity with plain-data components, but a wall, machine, storage tile, or resource source is not an ECS entity. It is a grid cell. Systems such as execution bridge the two: an entity's `ActionComponent` changes a `TileData` resource buffer and the inhabitant's inventory in the same causal operation. Calling the implementation simply "ECS" obscures this deliberate division.[6]

7.4 Indifferent Institution and Human Indirection

Implemented. Canonical `external.policy_variant = 1` applies stable physical rules to conveyor wear/load, storage occupancy, and anonymous occupancy capacity. It cannot inspect identity, personality, action, trust, opinion, graph community, or output semantics. Canonical `external.supply_variant = 1` links only shipped output to delayed material replenishment. The old strategic policy remains under variant 0 solely for controlled A/B comparison; it is not the canonical ontology of the factory.[7]

[6] See `src/components.h`, `src/grid.h`, `src/social.h`, and `src/chronicle.h`. EnTT architecture background is discussed by (Nystrom 2014).

[7] See `src/sim_policy.cpp`, `src/sim_space_policy.cpp`, and `config/default.toml`. `tests/verify_policy_audit.cmake` statically forbids behavioral and social dependencies in

Design objective. The institution may be demanding and materially harmful, but it is not a player that hates, classifies, or strategically targets people. The human Director likewise changes environmental conditions rather than issuing orders to inhabitants. These are separate boundaries: one governs autonomous institutional pressure, and the other governs player intervention.

7.5 Iteration, Counterfactuals, and Maturity

Background. Iterative development is valuable when each model extension is observable and can be compared with a baseline. Without a counterfactual, however, a visually plausible pattern may be only a direct consequence of a coefficient or map stamp.

Implemented. Batch metrics distinguish selection, target lookup, target arrival, and effective execution. The same seed and build replay deterministically. Mechanism toggles exist for social learning, place affinity, artifact effects, lifecycle processes, BUILD, supply variant, and institutional policy. The regular regression set uses seeds `0 1 2 3 7`, while macro-claims use twenty or more seeds. Director interventions have their own typed replay ledger.[8]

Design objective. Documentation uses three maturity levels:

1. **Implemented mechanism:** directly traceable to code, configuration, and a focused test.
2. **Design objective:** a normative property such as locality, indirection, or causal legibility.
3. **Hypothesis or result:** an aggregate statement requiring a defined metric, comparison, horizon, and uncertainty.

Not established. Segregation, artistic subculture, informal leadership, and free-riding have not passed that third standard. Their absence is a valid result and must not be repaired by introducing a categorical label or privilege merely to make the pattern visible.

8 Complementary Algorithms

The core systems covered in previous sections (CA, procedural generation, agents, pathfinding, physics, social simulation) form the primary architecture of a community simulation. Several additional algorithms are used to fill specific gaps: organic region boundaries, vegetation modeling, group behavior, pattern generation, opinion dynamics, and—connecting several of the above under one algebraic roof—tropical geometry.

---

the canonical policy.

[8] See `src/metrics.h`, `src/batch_main.cpp`, `tests/simulation_tests.cpp`, `tests/verify_metrics.cmake`, and `tests/verify_replay.cmake`.

8.1 Voronoi Diagrams for Region Boundaries

Voronoi diagrams partition a plane into regions based on proximity to a set of generator points:

$$\text{Region}(p) = \{x \in \mathbb{R}^2 : d(x,p) < d(x,q)\ \forall q \neq p\} \tag{29}$$

The diagram was formally described by Georgy Voronoi in 1908, though the concept of partitioning space by proximity predates him (Descartes used a similar construction in 1644 for mapping planetary regions). Voronoi diagrams have the property that all boundaries are equidistant from their nearest generators, producing convex polygons.

For world generation, Voronoi diagrams produce regions with organic, irregular boundaries suitable for biome delimitation, political territories, or cultural zones. The *relaxed* variant (Lloyd's algorithm: compute Voronoi, move each generator to its region's centroid, repeat) produces more uniform cell sizes and is used when approximately equal-sized regions are desired.

Computation of the Voronoi diagram for n points in 2D is $O(n \log n)$ via Fortune's sweep-line algorithm. For simulation purposes, the diagram is typically computed once during world generation and cached.

8.2 L-Systems for Vegetation Modeling

Lindenmayer systems (L-systems) are parallel rewriting grammars originally developed by Lindenmayer (1968) to model the growth patterns of filamentous organisms. An L-system consists of:

- An *alphabet* Σ of symbols.
- An *axiom* $\omega \in \Sigma^+$ (the initial string).
- A set of *production rules* $P \subset \Sigma \times \Sigma^*$, each mapping a symbol to a replacement string.

At each iteration, all symbols in the current string are replaced simultaneously by their production rules (parallel rewriting, distinguishing L-systems from sequential Chomsky grammars). When interpreted as turtle graphics commands (F = draw forward, $+$ = turn right by angle θ, $-$ = turn left, [= push state,] = pop state), the resulting strings produce branching structures.

Different rule sets and branching angles produce distinct morphologies: binary trees ($F \to F[+F]F[-F]F$ with $\theta \approx 25.7°$), bushes, ferns, coral structures, and algal colonies. The stochastic L-system variant assigns probabilities to multiple production rules for the same symbol, producing natural variation within a species.

For community simulation, L-systems are used during world generation to produce vegetation and, by extension, to model any growth process that follows branching patterns (rivers, cave systems, root networks). The computational

cost is $O(k^n)$ where k is the average rule length and n is the iteration count, but n is typically small (3–7 iterations).

8.3 Boids: Reynolds' Flocking Model

Reynolds (1987) introduced the Boids model to generate realistic flocking behavior for computer animation. Each agent (boid) computes its velocity based on three local forces derived from nearby neighbors within a perception radius:

- **Separation**: steer to avoid crowding neighbors.

$$\vec{F}_s = k_s \sum_{j \in N_i} \frac{\vec{p}_i - \vec{p}_j}{\|\vec{p}_i - \vec{p}_j\|^2}$$

- **Alignment**: steer towards the average heading of neighbors.

$$\vec{F}_a = k_a(\bar{\vec{v}}_{N_i} - \vec{v}_i)$$

- **Cohesion**: steer towards the average position of neighbors.

$$\vec{F}_c = k_c(\bar{\vec{p}}_{N_i} - \vec{p}_i)$$

The updated velocity is:

$$\vec{v}_{\text{new}} = \vec{v} + \vec{F}_s + \vec{F}_a + \vec{F}_c \tag{30}$$

Each force is weighted by a coefficient (k_s, k_a, k_c) that controls the relative strength of each behavior. The model produces collective motion that is qualitatively similar to biological flocking, schooling, and herding, despite having no centralized control and no explicit representation of the flock as a whole.

For community simulation, boids can control herd animal behavior, migration patterns, or crowd dynamics in settlements. The model is computationally $O(n^2)$ in the naive implementation (each agent checks all others), but spatial hashing reduces this to $O(n)$ average case by limiting neighbor queries to nearby cells.

8.4 Turing Patterns

Turing (1952) proposed that biological pattern formation (stripes, spots, spirals) can arise from the interaction of two chemical morphogens with different diffusion rates:

$$\frac{\partial u}{\partial t} = D_u \nabla^2 u + f(u, v) \tag{31}$$

$$\frac{\partial v}{\partial t} = D_v \nabla^2 v + g(u, v) \tag{32}$$

where u is the activator, v is the inhibitor, $D_v \gg D_u$, and f, g are nonlinear reaction terms (typically activator-inhibitor kinetics). The mechanism works because the activator creates local positive feedback (short-range activation) while the inhibitor suppresses activation at distance (long-range inhibition). The resulting instability (the Turing instability) spontaneously breaks spatial symmetry, producing stable, periodic patterns.

The specific pattern (spots, stripes, labyrinths, hexagons) depends on the ratio D_v/D_u, the reaction kinetics, and the domain geometry. Turing patterns have been verified experimentally in chemical systems (the Belousov-Zhabotinsky reaction, the CIMA reaction) and are hypothesized to play a role in biological pattern formation, though direct in vivo confirmation remains limited.

For world generation, Turing patterns can generate spatial variation in biome properties, resource distribution, or terrain texture. The discrete implementation on a grid is straightforward: approximate the Laplacian ∇^2 by the finite difference:

$$\nabla^2 u_{i,j} \approx u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}$$

and iterate the coupled equations with forward Euler time-stepping.

8.5 Opinion Dynamics: Modeling Belief Propagation

A social phenomenon that is underexplored in simulation games but well-studied in the mathematical social sciences is the dynamics of opinion formation. Two canonical models provide computationally efficient formalisms.

The **DeGroot model** (1974) describes opinion convergence in a network. Each agent i holds an opinion $x_i \in \mathbb{R}$ and updates it as a weighted average of its neighbors' opinions:

$$x_i^{(t+1)} = \sum_j w_{ij} x_j^{(t)}$$

where $w_{ij} \geq 0$ and $\sum_j w_{ij} = 1$. The weights w_{ij} represent the trust or influence that agent i accords to agent j. Under mild conditions (strongly connected, aperiodic influence graph), all opinions converge to a consensus value that is a weighted average of initial opinions. The rate of convergence depends on the spectral gap of the weight matrix.

The **bounded confidence model** (Hegselmann and Krause, 2002) adds a homophily threshold ϵ: agent i only averages the opinions of neighbors whose opinions are within ϵ of its own:

$$x_i^{(t+1)} = \frac{1}{|N_i(\epsilon)|} \sum_{j:|x_j - x_i|<\epsilon} x_j^{(t)}$$

This small modification fundamentally changes the dynamics. Instead of convergence to a single consensus, the population fragments into clusters separated by at least ϵ. The number and size of clusters depend on ϵ and the initial opinion distribution. For small ϵ, many small clusters form (extreme polarization); for large ϵ, the population converges (broad consensus).

For community simulation, opinion dynamics can model the spread of cultural traits, political beliefs, social norms, or technological preferences through a population. The computational cost is $O(V + E)$ per tick (each edge contributes to one update), which is negligible compared to pathfinding or fluid simulation. The emergent phenomena—consensus, polarization, echo chambers, cultural fragmentation—are produced by simple averaging operations without any scripted narrative.

8.6 Tropical Geometry: The Algebraic Shape of Decision, Search, and Utility

The preceding subsections present heterogeneous algorithms (spatial tessellation, generative grammars, flocking, reaction-diffusion, opinion averaging) that appear mathematically unrelated. Two load-bearing computations in *La Vida Misma* have a useful tropical interpretation: shortest-path search is min-plus, and greedy action selection is the zero-temperature limit of the implemented Boltzmann selector. Some min/max utility fragments can also be described tropically, but the complete nonlinear utility model cannot.

8.6.1 The Tropical Semiring

The **tropical semiring** (also called the min-plus or max-plus semiring) is the set $\mathbb{R} \cup \{\infty\}$ equipped with two operations:

$$a \oplus b = \min(a, b), \qquad a \otimes b = a + b$$

where the tropical "addition" $\oplus$ is ordinary minimum and the tropical "multiplication" $\otimes$ is ordinary addition. The additive identity is ∞ (the annihilator of min) and the multiplicative identity is 0. The dual **max-plus** convention takes $\oplus = \max$ instead. Either convention yields a semiring; crucially, $\oplus$ is **idempotent** ($a \oplus a = a$), which is the property that makes tropical geometry behave differently from classical linear algebra.

8.6.2 Application 1: A* Pathfinding Is Min-Plus Linear Algebra

The shortest-path problem is the canonical computation of the min-plus semiring (Mohri 2002). Pathfinding via A* (section 4) is a concrete instance: the

total path cost is the tropical product (ordinary sum) of edge weights, and the optimal path is the tropical sum (ordinary minimum) over alternative paths.

The implementation in `pathfinding.h` makes this literal. The `f`-score is the tropical product of the accumulated cost and the heuristic:

```
struct Node { int g; int f; /* g + heuristic */ };
```

Path relaxation accumulates cost via ordinary `+` (tropical $\otimes$) and selects via strict `<` (tropical $\oplus = \min$):

```
int ng = cur.g + 1;                  // tropical product
if (ng < g_score[ni]) {              // tropical sum (min)
    g_score[ni] = ng;                // write the new minimum
    int nf = ng + |dx| + |dy|;       // tropical product with heuristic
}
```

The initial `g_score` seed of `999999` is the additive identity ∞ of the min-plus semiring. The min-heap keyed on `f` repeatedly extracts the min of the open set. This is a textbook fixpoint computation over $(\mathbb{R} \cup \{\infty\}, \min, +)$.

8.6.3 Application 2: Boltzmann Selection and the Max Limit

The greedy limiting rule $a^* = \arg\max_a U(a)$ is exactly the $\oplus = \max$ operation of the max-plus semiring. The implemented agent selector in `sim_utility.cpp` instead uses a **Boltzmann softmax** over feasible positive-score actions, controlled by a temperature τ:

```
float tau = config_.selection_temperature;     // config.h: tau=0.4, "0=greedy"
weights[i] = std::exp((u - max_u) / tau);      // sim_utility.cpp
```

After normalization, these weights define softmax probabilities. The related scalar LogSumExp function is

$$\mathrm{LSE}_\tau(\mathbf{u}) = \tau \log \sum_i e^{u_i/\tau} \xrightarrow{\tau \to 0^+} \max_i u_i.$$

Its gradient with respect to utilities is the softmax distribution. As temperature approaches zero, probability concentrates on maximizing actions; it becomes one-hot only when the maximum is unique, while ties retain mass across maximizers. At high temperature the implementation approaches a uniform distribution over positive-score feasible actions. These are limiting facts about stochastic choice, not an interpolation between semirings. A structurally similar target softmax survives only in historical `external.policy_variant = 0`; the canonical indifferent institution uses a deterministic physical maximum and must not be described as a second strategic selector.

8.6.4 Application 3: Limits of the Utility Analogy

A function assembled solely from finitely many affine pieces through `min`, `max`, and addition admits a tropical-polynomial or tropical-rational description. Some small utility fragments have that shape, such as clamped linear factors and maxima between action sub-scores. The implemented action utilities as a whole do not.

Canonical survival urgency includes a sigmoid; retained variants include powers and quadratic spikes; Bonabeau response thresholds are rational functions; and many terms are multiplied together. Threshold branches may add kinks or jumps, but they do not turn these nonlinear expressions into affine pieces. Results for piecewise-linear classifiers and tropical hypersurfaces (Zhang et al. 2018) therefore cannot be transferred to the full behavioral surface without first constructing and validating an explicit piecewise-linear approximation.

8.6.5 Why This Unification Matters

Keeping exact tropical structure separate from limiting analogies has three payoffs:

- **Precision.** Path cost is genuinely min-plus, while temperature describes a smooth approximation to max. The analogy relates their limiting operations but does not transfer convergence theorems between search and stochastic choice.
- **Historical comparison.** Noel et al. (2013) define *tropical equilibration* as the regime where two opposing monomials have equal magnitude. The retained legacy `adversary_intensity` and `restructure_temperature` controls can be studied through that lens, but they do not describe canonical `external.policy_variant = 1`.
- **Auditability.** Tropical tools may analyze explicitly piecewise-linear approximations or isolated min/max fragments. They do not currently provide a count of behavioral regions for the nonlinear executable utility.

This subsection is descriptive rather than prescriptive: it does not change the implementation. It identifies exact tropical structure in pathfinding, a limiting connection in action selection, and the boundary beyond which that framing no longer describes the executable utility.

9 Reference Technology Roadmap (Non-Executable)

> **Maturity:** This section is a general reference architecture assembled from the literature, not the implementation roadmap or current feature list for *La Vida Misma*. The executable source of truth is section 18 and `doc/design_spec.md`; the active project roadmap is `ROADMAP.md`. The phases and deliverables below are design possibilities and must not be read as present-tense claims about the

repository.

This section provides a hypothetical phased approach to building a broader community simulation engine. The roadmap is ordered by dependency: each phase would produce a testable system, and each subsequent phase would depend on the previous one.

9.1 Component Summary

Component	Algorithm	Key equations	Complexity per tick
Terrain	Perlin/Simplex + fBm	equation (11)	$O(M)$ at generation; $O(1)$ per query
Biomes	Multi-field thresholds	Classification rules	$O(M)$ at generation
Regions	Voronoi + Lloyd relaxation	equation (29)	$O(n \log n)$ at generation
Rivers	Gradient descent on elevation	Optimization	O(river cells) at generation
Fluids	3D cellular automaton	equation (21), equation (22)	$O(F)$ where $F =$ fluid tiles
Pathfinding	A* + connected components + cache	equation (20)	$O(b^d)$ per query; amortized $O(1)$ with cache
Agent AI	Utility theory + needs	equation (14), equation (17)	$O(\|A\| \cdot \|a\|)$ per agent
Personality	Facet vector + stress layers	equation (19)	$O(\|\text{events}\|)$
Storyteller	Temporal distribution	equation (28)	$O(1)$ per evaluation
Social graph	Weighted edges + contagion	equation (18), equation (27)	$O(V + E)$
Spatial games	Grid-based game theory	(Nowak and May 1992; Schelling 1971)	$O(\|\text{agents}\|)$
Structures	WFC or graph grammar	equation (13)	$O(\|\text{tiles}\| \cdot \|\Sigma\|)$
Vegetation	L-systems	Rewrite grammar	$O(k^n)$ at generation
Heat	Diffusion on grid	equation (23)	$O(M)$
Opinion dynamics	DeGroot / bounded confidence	Weighted averaging	$O(V + E)$

Note: M = total map tiles, F = fluid tiles, $|A|$ = number of agents, $|a|$ = number of actions per agent, V = vertices, E = edges in social graph, Σ = tile state set.

9.2 Phase 1: Terrain and Biome Generation

Implement Perlin/Simplex noise with multi-octave fBm (equation (11)). Generate an elevation map. Add temperature (function of latitude + elevation), rainfall (noise + orographic shadow), and drainage (separate fractal). Classify biomes via threshold intersection of all layers. Generate rivers via gradient descent on the elevation map. Delimit regions via relaxed Voronoi.

Deliverable: A reproducible world map with internally consistent geography. The biomes form at the intersection of independent layers, and the geography is deterministic given the random seed.

Estimated scope: ~2,000 LOC. Key data structures: 2D/3D arrays for terrain layers, noise function implementations, biome classification table.

Primary risk: Tuning the biome threshold values requires iterative experimentation. There is no analytical method for determining "correct" thresholds; the criterion is whether the resulting geography appears plausible when inspected visually.

9.3 Phase 2: Agents, Needs, and Utility AI

Implement ECS architecture. Create agents with: Position component, Needs component (hunger, rest, social, safety), Personality component (facet vector), and a UtilityAI system that evaluates available actions. Actions include: move to location, eat, sleep, socialize, work. Implement need decay (linear or exponential) and urgency curves (equation (17) with $\alpha = 1.5$ as a starting point).

Deliverable: Agents that autonomously prioritize actions based on their internal state, with personality variation producing different behavioral patterns across agents.

Estimated scope: ~3,000 LOC. Key data structures: ECS framework with component pools, utility evaluation loop.

Primary risk: Balancing utility weights across needs. If one need consistently dominates, all agents will exhibit identical behavioral patterns. The weight structure must allow for personality-driven differentiation.

9.4 Phase 3: Pathfinding and Navigation

Implement A* with an appropriate heuristic for the terrain grid. Add connected components tagging via flood-fill (so impossible paths abort without search). Add LRU path cache. Add configurable tile costs (agents avoid difficult terrain unless necessary).

Deliverable: Agents navigate efficiently around obstacles. Path computation is fast enough for 50+ simultaneous agents requesting paths every few ticks.

Estimated scope: ~1,500 LOC. Key data structures: priority queue, component index for spatial queries, path cache.

Primary risk: Cache invalidation when the map changes (construction, destruction, flooding). An overly aggressive invalidation strategy negates the cache benefit; an overly conservative one produces stale paths.

9.5 Phase 4: Physical Systems (Fluids, Heat, Collapse)

Implement 3D CA fluids (7-level system per equation (21) and equation (22), plus pressure via flood-fill). Implement heat diffusion (equation (23) with per-material conductivity). Implement structural collapse (connected-to-surface check via flood-fill).

Deliverable: Water flows, heat spreads, unsupported structures collapse. These systems interact through the shared tile grid.

Estimated scope: ~2,000 LOC. Key data structures: 3D CA grid for fluids, material property tables, flood-fill queue.

Primary risk: Fluid pressure propagation is the most computationally expensive component. Naive flood-fill on every tick for every fluid source is $O(M)$; optimization requires identifying connected fluid bodies and updating them incrementally.

9.6 Phase 5: Economy (Production Chains and Skills)

Implement a production graph: raw resources enter as inputs to workshops, which produce intermediate goods and final products. Agents gain XP for completed tasks (equation (25)). Skill level affects output quality via quality tier probabilities. Distance reduces task utility (equation (24)), creating natural industrial organization.

Deliverable: A self-organizing economy where agents specialize through practice, workshops cluster around resource sources, and resource scarcity drives behavioral adaptation.

Estimated scope: ~1,500 LOC. Key data structures: production graph, skill tables, quality probability tables.

Primary risk: Balancing XP curves and quality thresholds so that skill progression feels meaningful but not grind-intensive. The mathematical model provides the framework, but the specific numerical values require playtesting.

9.7 Phase 6: Society (Social Graph, Stress, Spatial Games)

Implement a relationship graph with affinity edges in $[-100, 100]$. Events modify edges (equation (18)). Stress accumulates (equation (19)) with personality modifiers and three-layer memory. Stress contagion propagates through the relationship graph (equation (27)). Implement spatial game-theoretic interactions: agents compete for resources based on spatial proximity, and cooperation clusters form naturally per the Nowak-May model (Nowak and May 1992).

Deliverable: Communities with internal politics, friendships, rivalries, and cascading stress events. Two agents who experience the same event react differently based on personality and relationship history.

Estimated scope: ~2,500 LOC. Key data structures: adjacency graph, event queue, stress accumulator.

Primary risk: Stress cascade tuning. If contagion is too strong, a single negative event collapses the entire community; if too weak, social dynamics are invisible.

9.8 Phase 7: Meta-Agent and Opinion Dynamics

Implement a Storyteller meta-agent that calibrates external pressure (threats, events, resource scarcity) based on colony wealth and elapsed time (equation (28)). Implement opinion dynamics (DeGroot or bounded confidence models) so cultural traits and beliefs evolve through agent interaction.

Deliverable: A simulation that generates narrative rhythm (calm periods followed by pressure) and evolving cultural landscapes.

Estimated scope: ~1,000 LOC. Key data structures: distribution sampler for event timing, opinion vectors per agent.

Primary risk: The Storyteller difficulty curve is entirely empirical. There is no theoretical basis for the function parameters; they must be calibrated to produce engaging pacing.

9.9 Estimated Total Scope

Phase	Approximate LOC	Cumulative
1. Terrain + Biomes	2,000	2,000
2. Agents + Needs + AI	3,000	5,000
3. Pathfinding	1,500	6,500
4. Physics	2,000	8,500
5. Economy	1,500	10,000
6. Society	2,500	12,500
7. Storyteller + Opinions	1,000	13,500

The total estimate of ~13,500 LOC is conservative compared to Dwarf Fortress (~700,000 LOC over 20 years). The difference reflects scope: this estimate covers a functional engine that demonstrates the core emergent phenomena described in this document, not the full content depth of a commercial game.

9.10 Deliberately Excluded (Initial Scope)

- **Rendering / graphics**: Console/terminal output is sufficient for validating simulation behavior.
- **Networking / multiplayer**: Single-threaded, local simulation.
- **Save/load serialization**: Can be added as a serialization layer over the ECS state.
- **Machine learning**: The neural CA results (Section 1.7) are relevant to long-term research but premature for an initial implementation. Handcrafted utility functions are sufficient.
- **Navier-Stokes fluid dynamics**: The 7-level CA approximation produces adequate behavioral fidelity for community simulation purposes at a fraction of the computational cost.

10 Verified Academic References

The following references were verified through their official sources (arXiv, publisher DOI, or institutional repositories) and form the academic backbone of this document. Each entry includes the formal citation, a summary of the work, and its relevance to the simulation framework discussed here.

10.1 Cellular Automata and Continuous Generalizations

Lenia: Biology of Artificial Life (Chan 2019) arXiv:1812.05433. Bert Wang-Chak Chan, published in *Complex Systems*, Vol. 28, No. 3, 2019. A continuous cellular automaton framework with continuous space, time, and state. The evolution equation $dA/dt = G(K * A^t)$ generalizes the discrete CA update to a continuous domain. By varying the growth function G and kernel K, Chan identified over 400 self-organizing pattern types ("species") classified into 18 families. The work demonstrates that CA-like emergence is not an artifact of discretization but a property of the underlying mathematical structure.

Discretization and Self-Organization in Continuous CA (Davis 2022) arXiv:2208.09444. Q. Tyrell Davis, University of Vermont, 2022. Demonstrates that discretization artifacts are essential for self-organization in Lenia. The glider *Scutium gravidus* destabilizes when numerical precision is increased (float32 to float64), when spatial resolution is increased (larger kernels), or when temporal resolution is increased (smaller time steps). The result suggests that emergent patterns in numerical simulations are adapted to the computational substrate on which they are discovered, with implications for the choice of numerical representation in simulation engines.

SmoothLife: Generalization of Conway's Game of Life (Rafler 2011) arXiv:1111.1567. Stephan Rafler, 2011. Replaces binary cell states with continuous values and hard thresholds with sigmoid functions. Introduces inner/outer filling integrals over annular regions and continuous time-stepping via differential equations. Produces translating structures ("smooth gliders") that propagate in arbitrary directions, demonstrating that GoL's emergent properties do not depend on spatial discretization.

Image Segmentation via Cellular Automata (Sandler et al. 2020) arXiv:2008.04965. Sandler, Zhmoginov, Luo, Mordvintsev, Randazzo, and Agüera y Arcas (Google AI), 2020. Formulates the CA update rule as a learnable 3-layer neural network with residual connections ($< 10{,}000$ parameters). Demonstrates that purely local information exchange can solve image segmentation, a task that appears to require global context. Reports "regime change" transitions during training (abrupt stability shifts analogous to phase transitions). Establishes a connection between CA theory and deep learning.

10.2 Emergence and Complexity

Computation at the Edge of Chaos (Langton 1990) Chris Langton, *Physica D* 42, 1990. Introduces the λ parameter to quantify the location of the "edge of chaos" in CA rule space. Demonstrates that computational capability peaks between ordered (Class I/II) and chaotic (Class III) regimes. Provides the theoretical basis for the observation that interesting emergent behavior requires rules calibrated at a critical boundary.

Universality and Complexity in Cellular Automata (Wolfram 1984) Stephen Wolfram, *Physica D* 10, 1984. Establishes the four-class taxonomy of cellular automata behavior: fixed point (I), periodic (II), chaotic (III), and complex localized structures (IV). Class IV is identified as the regime where computation-like behavior and glider-like structures emerge. Foundational classification still in use in CA research.

10.3 Agent-Based Modeling and Social Simulation

Growing Artificial Societies (Epstein and Axtell 1996) Joshua Epstein and Robert Axtell, MIT Press / Brookings Institution Press, 1996. The Sugarscape model: autonomous agents on a 2D grid harvest resources, reproduce, trade, and fight under simple local rules. Demonstrated emergent wealth inequality (Gini ~ 0.5), cultural differentiation, combat, and trade networks. Foundational text for agent-based modeling in social science.

Dynamic Models of Segregation (Schelling 1971) Thomas Schelling, *Journal of Mathematical Sociology*, 1971. Agents with mild preference for similar neighbors (threshold as low as 30%) spontaneously produce highly segregated neighborhoods on a 2D grid. Demonstrates that individual micromotives can produce collective macro-outcomes that no individual intended. One of the most

influential results in agent-based social modeling.

The ODD Protocol for Agent-Based Models (Grimm et al. 2020) Volker Grimm et al., *Journal of Artificial Societies and Social Simulation*, 2020. A standardized protocol (Overview, Design, Details) for describing agent-based models. Ensures models are replicable and comparable. Provides a disciplined framework for simulation design with explicit specification of entities, state variables, process scheduling, and submodel mathematics.

10.4 Game Theory and Spatial Dynamics

Evolutionary Games and Spatial Chaos (Nowak and May 1992) Martin Nowak and Robert May, *Nature* 359, 1992. Demonstrates that placing Prisoner's Dilemma on a spatial lattice fundamentally changes evolutionary outcomes: cooperators form clusters that resist defector invasion, producing spatial chaos and fractal patterns. The spatial structure itself enables cooperation that is impossible in well-mixed populations. Directly relevant to how spatial layout affects social dynamics in community simulation.

10.5 Procedural Generation

WaveFunctionCollapse (Gumin 2017) Maxim Gumin, GitHub repository, 2016–2017. A constraint satisfaction algorithm for generating tile maps that satisfy adjacency constraints. Uses Shannon entropy as a variable-ordering heuristic to select the next cell to collapse. Operates in tile-based mode (explicit constraints) and overlapping mode (constraints learned from exemplar images). Widely adopted in procedural generation for games.

Analysis of WFC as CSP (Karth and Smith 2017) Isaac Karth and Adam Smith, *Proceedings of the 12th International Conference on the Foundations of Digital Games*, 2017. Formalizes WFC as constraint satisfaction with arc consistency checking and minimum-remaining-values variable ordering. Connects the algorithm to the academic CSP literature and provides theoretical grounding for its effectiveness.

10.6 Pathfinding

The JPS Pathfinding System (Harabor and Grastien 2012) Daniel Harabor and Alban Grastien, *Proceedings of the 5th Symposium on Combinatorial Search (SOCS)*, 2012. Accelerates A* on uniform-cost grids by identifying "jump points"—nodes where the optimal path must turn—and skipping intermediate nodes. Reduces expanded nodes by up to 100x compared to standard A* on uniform grids. Limited to uniform-cost grids; does not apply to variable-cost terrain.

11 Part II: Design Specification — La Vida Misma

Part I of this document established the mathematical foundations of community simulation: cellular automata, procedural generation, agent-based modeling, utility theory, pathfinding, physical systems, social simulation, and implementation methodology. Part II applies these foundations to a specific design: *La Vida Misma*, a community simulation where agents inhabit a factory they did not build, do not control, and do not understand.

The central design tension is between two forces acting on each agent: the *factory's requirements* (survival, production, compliance) and the *agent's internal drives* (expression, social connection, rest, purpose). The factory demands output. The agents want to live. The simulation explores what happens when these forces interact on a shared spatial substrate.

12 The Factory: World Substrate and Institution

The factory has two compatible descriptions. As a **substrate**, it is the grid, resources, machines, storage, conveyors, Entrance, and Exit. As an **institution**, it imposes an output demand and updates material support through anonymous rules. It is not implemented as a strategic player in a zero-sum game.

12.1 Grid and Tile Model

Implemented. The canonical configuration creates a bounded 60×40 grid. Tile type and mutable `TileData` are stored in dense arrays, while inhabitants and artifacts are EnTT entities. This is the hybrid ECS/Grid architecture described in section 7.

Tile type	Current role
`Floor`	ordinary buildable and usable space
`Wall`	the only non-walkable tile type
`Machine`	built or incomplete Food, Materials, or Output machine
`Storage`	shared buffer for all five resources
`Exit`	boundary associated with the sole institutional output drain
`Entrance`	boundary at which exogenous arrivals enter
`OpenSpace`	spatial label without an exclusive cultural-action privilege

Tile type	Current role
`FoodSource`	renewable `RAW_FOOD` deposit and FoodMachine site
`ScrapPile`	renewable `RAW_MATERIAL` deposit and MaterialsMachine site
`EatingZone`	inherited or buildable label without canonical satisfaction or capacity bonus
`Conveyor`	directed, degrading, single-resource transport segment
`HiddenSpace`	discoverable place; canonical closure uses anonymous occupancy capacity rather than cultural meaning

All non-wall tiles, including built conveyors and machines, are currently walkable. Earlier descriptions of active belts as impassable are obsolete. Agent movement uses cardinal cached A*, and a tile can hold at most six moving inhabitants.[9]

Resource tiles and source-backed machines retain an amount, maximum, and base regeneration rate. Storage has a shared capacity and separate buffers for the five resources. Conveyors retain direction, condition, maintenance priority, one content type, and one content amount. A condition below 0.2 stops transport; `MAINTAIN` can restore a degraded belt.

12.2 Inherited Three-Machine, Two-Branch Factory

Implemented. The generated world contains an operational but degraded factory before the initial population appears. For every one of twenty tested seeds, the map contains one built machine of each subtype, at least three built storages, one Entrance, one Exit, and four built output conveyors with initial conditions 0.55, 0.65, 0.75, and 0.85. The OutputMachine is connected to the Exit-side storage, and all three machines are reachable. Inherited structures are counted separately from later construction by inhabitants.[10]

The three machines form two physical branches:

1. **Food branch:** `FoodSource` → FoodMachine → processed `FOOD`. A source-backed FoodMachine regenerates and auto-gathers `RAW_FOOD`; effective `WORK` consumes that input. Part of the result can remain with the worker and the remainder is deposited into nearby storage or a conveyor.

[9] Implementation references: `src/grid.h`, `src/components.h`, `src/sim_movement.cpp`, and `[grid]` in `config/default.toml`.

[10] `src/wfc_generator.h` stamps the inherited chain. `test_inherited_factory_map_properties` and `test_inherited_chain_operates_without_build` in `tests/simulation_tests.cpp` verify twenty generated maps and no-BUILD operation.

2. **Institutional output branch:** `ScrapPile` → MaterialsMachine → `CONSTRUCTION_MATERIAL` → OutputMachine → `OUTPUT` → Exit-side storage → shipment. The MaterialsMachine is source-backed, produces construction material, and recycles part of its byproduct into nearby scrap deposits. The OutputMachine consumes construction material. Output reaches the drain only by a directed conveyor route or by an inhabitant hauling it to storage within Manhattan radius three of the Exit.

Manual `GATHER`, hauling, additional construction, repair, rerouting, and storage remain useful, but `BUILD` is not the founding act of the colony. A focused fixture operates all three inherited machines and ships output without selecting or executing `BUILD`.

12.3 Resources and Logistics

Implemented. The resource set is exactly:

Resource	Main production or use
`RAW_FOOD`	gathered or auto-gathered from FoodSource; FoodMachine input; edible raw with disease risk
`RAW_MATERIAL`	gathered or auto-gathered from ScrapPile; MaterialsMachine and construction input
`FOOD`	processed nutrition consumed by inhabitants
`CONSTRUCTION_MATERIAL`	MaterialsMachine output; input to OutputMachine operation and construction
`OUTPUT`	OutputMachine product; the only resource that satisfies institutional demand after shipment

A conveyor carries one resource type at a time and rejects a different type until emptied. Contents move at configured throughput toward another conveyor, a compatible machine buffer, storage, or the Exit-side storage. Output in a remote storage, machine buffer, inventory, or disconnected belt is physically present but institutionally unshipped. Partial deposits preserve the undelivered remainder.[11]

The canonical configuration uses renewable FoodSource and ScrapPile deposits. Their effective regeneration is the sole variable controlled by delayed external support. They are not accurately described as fixed finite budgets.

[11] See `src/recipes.h`, `src/sim_execute.cpp`, and `src/sim_conveyor.cpp`. Focused tests cover single-type belts, direct machine feeding, required hauling arrival, and partial deposits.

12.4 Shipped Output and Delayed Material Support

Implemented. Let q_t be demand during tick t, let y_t be output actually drained from Exit-side storage, and define

$$f_t = \begin{cases} \text{clamp}(y_t/q_t, 0, 1), & q_t > 0, \\ 1, & q_t = 0. \end{cases}$$

External support is an exponential moving average,

$$s_t = s_{t-1} + \left(1 - e^{-1/T}\right)(f_t - s_{t-1}), \tag{33}$$

with canonical response time $T = 600$ ticks. Support is mapped to a regeneration factor by

$$r_t = r_{\min} + (1 - r_{\min})\,\text{smoothstep}\left(\text{clamp}\left(\frac{s_t - s_{\text{low}}}{s_{\text{high}} - s_{\text{low}}}, 0, 1\right)\right), \tag{34}$$

where `supply_floor = 0.20`, `supply_low = 0.05`, and `supply_high = 0.45`. The factor multiplies only FoodSource and ScrapPile regeneration. Regeneration precedes shipping in the tick pipeline, so the support update from tick t affects replenishment no earlier than tick $t + 1$.

Only **shipped** `OUTPUT` drives this delayed support. Producing output, storing it, or moving it near the Exit without allowing the drain does not count. Missed demand does not directly change utility, stress, machine efficiency, or mortality under canonical `external.supply_variant = 1`. Hunger, exhaustion, breakdown, suicide, and natural mortality remain the death mechanisms; there is no canonical "factory health equals zero, therefore everyone dies" rule.[12]

`factory_health` remains as an aggregate condition diagnostic: built machines contribute one and built conveyors contribute their condition. In the canonical supply variant it is not the moral score or survival cause described by older drafts.

12.5 Indifferent Canonical Policy

Implemented. `external.policy_variant = 1` is canonical. At configured restructure epochs, a seed-, epoch-, and position-derived stable hash gates the event. Candidate priorities use only anonymous physical state:

- a built conveyor may be selected from wear and current load, with stable physical jitter;

[12] See `Simulation::system_ship_out_food()` and `Simulation::system_regen_resources()` in `src/simulation.cpp`, plus `[external]` in `config/default.toml`. `test_external_supply_causality` and `tests/verify_metrics.cmake` check the active variant and causal metrics.

- a nonempty storage may be selected from total occupancy fraction, with stable physical jitter.

The highest-priority conveyor loses 0.15 condition without crossing the repairable floor 0.20, or the highest-priority storage loses the same 10% fraction of each stored resource. This restructure rule does not read the agent registry. A separate overcapacity rule counts living positions anonymously; it may read only `alive` and position, never agent IDs, actions, behavioral targets, personality, opinions, trust, noncompliance, graph communities, utility, output stock as a special category, or behavioral RNG.[13]

`external.policy_variant = 0` preserves the historical strategic policy and its legacy social sanctions only for explicit A/B comparison. It is not the default and must not be used to explain the canonical factory as an adversarial mind. In `CALM`, quota demand, supply contraction, restructuring, deterioration, Watcher logic, and closure pressure are disabled.

12.6 Claims and Open Questions

Design objective. The inherited layout should make dependence material and repairable: sustained shipping supports future inputs, while sustained blockage contracts them gradually and leaves a nonzero recovery floor. The Director can alter physical conditions without commanding inhabitants (section 13).

Established result. In a twenty-seed, 3000-tick blocked-Exit comparison with restructuring disabled, blocking shipment reduced survivors in 14/20 seeds and increased starvation deaths in 18/20. In the recorded reopening experiment, support recovered before stocks, and survivor counts were better in four of five seeds with one tie. These results support the delayed material pathway at the tested horizons; they do not imply that every seed must respond monotonically.

Not established. The grid and production chain do not by themselves establish territories, segregation, public-goods equilibria, free-riding, or collective leadership. Those are separate empirical questions, not consequences that may be inferred from using a two-dimensional lattice.

13 The Director: Player as Environment Architect

The Director is the human intervention role. It is distinct from the autonomous institutional policy and from the configuration enum `DirectorMode`. The player changes environmental and institutional parameters; inhabitants continue to choose their own actions through the utility pipeline.

[13] See `src/sim_policy.cpp` and `src/sim_space_policy.cpp`. `test_indifferent_policy_ignores_social_state`, `test_indifferent_storage_policy_is_resource_neutral`, and `tests/verify_policy_audit.cmake` enforce the boundary.

13.1 Typed Intervention Boundary

Implemented. `DirectorCommand` is a closed variant with exactly five command types:[14]

Command	Accepted parameters	Effect and constraints
`DirectorSetQuota`	nonnegative finite output per tick	fixes subsequent demand; explicitly rejected in `CALM`
`DirectorSetZone`	grid coordinate and occupancy capacity $0 \dots 8$	applies only to `Floor` or `OpenSpace`; this is anonymous physical capacity, not a profession or cultural zone
`DirectorPlaceStructure`	coordinate, structure type, and machine subtype or conveyor direction where relevant	places a completed Wall, Storage, Food/Materials/Output Machine, or Conveyor on a compatible physical site
`DirectorRemoveStructure`	coordinate	removes Wall, Storage, Machine, or Conveyor; Entrance and Exit are protected
`DirectorSetMaintenancePriority`	conveyor coordinate and `Normal` or `High`	changes an observable priority signal but does not repair the belt or select a worker

FoodMachine placement requires a FoodSource, MaterialsMachine placement requires a ScrapPile, and OutputMachine, Wall, Storage, and Conveyor placement require Floor. Removing a source-backed machine restores the underlying resource deposit. Removing storage or a loaded conveyor records discarded contents as physical loss.

Implemented invariant. No Director command accepts an agent identity, action, behavioral target, personality, opinion, relationship, community, or utility. Static tests inspect both the public type boundary and its implementation for those forbidden dependencies. The Director can make an action more attractive or feasible by changing the world, but cannot make an inhabitant perform it.

[14] `src/director.h` defines the five-command variant; `src/sim_director.cpp` validates and applies it. The boundary audit is in `tests/verify_policy_audit.cmake`.

13.2 Indirection

Design objective. Player influence should remain environmental. A maintenance priority can raise the utility of repairing a visible belt, a storage placement can shorten a physical transfer, and a quota can alter institutional demand. None of these interventions identifies who must respond or guarantees that anyone will.

This is narrower than the construction and designation interfaces described in older drafts. There is no implemented command for assigning a workshop, residential district, artistic quarter, job, schedule, or individual order. `DirectorSetZone` means only sustained occupancy capacity, and canonical closure reads occupants at a coordinate without consulting their identities or culture.

13.3 Player View and Debug View

Implemented. The GUI starts in a player-facing view. Its panel exposes observable institutional and environmental consequences such as demand and fill, stocks, output flow, infrastructure condition, density, occupancy zones, maintenance priority, and factual events. Pressing `E` opens the five Director tools and pauses the simulation while editing.[15]

Exact needs, personality facets, directed relationships, and per-action utility are debug information rather than assumed player knowledge. `F12` explicitly toggles the debug view; `--debug` may also start the GUI there. This separation is epistemic as well as visual: internal values can be inspected for model diagnosis without being presented as information on which ordinary Director play is based.

13.4 Recording and Replay

Implemented. Every accepted intervention is recorded at the simulation tick before `Simulation::advance()` and receives a strictly increasing global sequence number. Invalid commands are atomic and do not enter the ledger.

`vida_gui --seed N --record FILE` writes a TOML log with format `vida-interventions`, schema version 2, seed, Director mode, `tick_phase = "before_advance"`, and an FNV-1a fingerprint of the loaded configuration source. `vida_batch replay <ticks> <seed> <file>` rejects an incompatible header, seed, configuration fingerprint, ordering, tick, command, or parameter, and applies accepted events at their exact recorded phase.[16]

The round-trip test compares the original session and replay across the Director ledger, metrics, full grid state, population, quota, and Chronicle. The CLI test

[15] See `src/main_gui.cpp`, `src/graphical_view.h`, and `src/graphical_view.cpp`.

[16] Serialization is in `src/director.cpp`; batch replay is in `src/batch_main.cpp`. `test_director_environmental_commands` and `test_director_log_round_trip_and_replay` in `tests/simulation_tests.cpp`, together with `tests/verify_replay.cmake`, cover the contract.

also requires byte-identical same-build replay output and rejects a mismatched seed or configuration source. Replay is therefore a current verification mechanism, not a future design proposal.

13.5 Scope of Claims

Implemented. The Director is a human environmental editor; the canonical factory policy is indifferent software. Neither is a RimWorld-style Storyteller that selects narrative events from inhabitant psychology. Chronicle stores factual events, while narrative rendering is behavior-neutral.

Not established. The available commands make controlled intervention and replay possible, but no experiment yet establishes that a particular Director strategy causes specialization, community formation, spatial segregation, or long-run survival. Those are outcome hypotheses and require comparative runs, not conclusions from the API design.

14 The Inhabitants: Current Agent Architecture

This section is an implementation account rather than an idealized utility model. Exact action scores remain in code because they are action-specific and combine urgency variants, feasibility, personality, mood, skill, local observations, social evidence, place affinity, and stress.

14.1 State and Motivation

Implemented. Each living inhabitant is an EnTT entity with position, needs, personality, action, stress, social state, opinions, inventory, skills, place memory, creative-work progress, lifecycle, and a private RNG component.[17]

`NeedsComponent` contains six decaying motivational deficits plus one condition variable:

Variable	Range and canonical update
`hunger`	$[0, 1]$; increases by 0.0035 per tick, amplified by disease
`rest`	$[0, 1]$; increases by 0.0040 per tick
`social`	$[0, 1]$; increases by 0.003 per tick
`expression`	$[0, 1]$; increases by 0.003 per tick
`purpose`	$[0, 1]$; increases by 0.002 per tick
`meaning`	$[0, 1]$; increases by a hard-coded 0.001 per tick and is reduced by completed creative work or discovery

[17] See `src/components.h`, `src/simulation.cpp`, and `src/sim_lifecycle.cpp`.

Variable	Range and canonical update
`disease`	$[0, 1]$; physical illness that recovers by 0.002 per tick and can multiply hunger decay up to 2.0

Canonical action effects include `eat_satisfaction = 0.022`, raw-food efficiency 0.7, rest recovery 0.020, social and creative satisfaction 0.012, explore purpose gain 0.008, and work purpose gain 0.004. Raw meals have an 8% per-meal disease chance and add 0.15 disease severity when eaten from personal inventory. A full processed portion eaten from personal inventory is free of that risk and accelerates recovery. The nearby-storage helper currently applies raw food's lower satisfaction but does not perform the disease roll or the processed food recovery step; this is an implementation asymmetry, not a claimed nutritional model.[18]

The six motivational deficits do not all enter stress in the same way. Social, expression, purpose, and meaning primarily affect utility and mood; canonical direct stress input comes from critical hunger, critical rest, and disease.

14.2 Personality and Opinions

Implemented. Personality has six continuous facets in $[0, 1]$: `compliance`, `laziness`, `artistry`, `gregariousness`, `resilience`, and `curiosity`. They modulate productive willingness, rest, creation, social contact, stress susceptibility, exploration, and action response thresholds. They are not job assignments.

Founders sample one of six archetype profiles and apply bounded jitter to produce their initial continuous traits and opinion priors. The archetype remains useful for display, but there is no fixed sequence by ID and no routing by archetype. Arrivals sample continuous traits independently. A child receives the mean of its parents' six traits plus bounded mutation of at most `0.08`; it does not inherit a founder archetype.

The separate `OpinionComponent` stores four continuous stances: work ethic, risk tolerance, tradition, and solidarity. During effective social contact, sufficiently close opinions can move through bounded-confidence, influence-weighted exchange. An opinion may cause local directed disapproval, but canonical institutional policy cannot read it.

14.3 Five Resources and Inventory

Implemented. The resource taxonomy has exactly five values: `RAW_FOOD`, `RAW_MATERIAL`, `FOOD`, `CONSTRUCTION_MATERIAL`, and `OUTPUT`. `InventoryComponent`

[18] Effective values are in `[needs]`, `[actions]`, `[disease]`, and `[stress]` of `config/default.toml`; header values in `src/config.h` are fallbacks and may differ.

has a slot for every one of them and total carrying capacity 10. The canonical processed-food pocket target is separately limited by `inv_food_cap = 2.0`; storage and machine buffers hold communal or in-process stock. Output carried by an agent has no institutional effect until deposited at Exit-side storage and shipped.

14.4 Four Skills

Implemented. Skills are `factory_work`, `domestic`, `artistic`, and `social_skill`, each derived from persistent XP and capped at level 5:

$$\text{level}(x) = \min(5, x/10). \tag{35}$$

An effective relevant action adds `0.1` XP: GATHER trains domestic, WORK trains factory, CREATE trains artistic, and SOCIALIZE trains social skill. There is no forgetting by disuse. Execution uses

$$\text{level\_bonus}(\ell) = 1 + 0.15\ell, \tag{36}$$

for the applicable output or satisfaction, while action utility receives a smaller $1 + 0.02\ell$ preference multiplier. Skill therefore creates a smooth practice-effectiveness-preference feedback, not a profession gate. Focused tests confirm that artistic and social skills now progress.[19]

14.5 Thirteen Actions

Implemented. `ActionType` has thirteen selectable values, including `IDLE`:

Action	Current effect or role
`GATHER`	collect visible raw food or raw material
`BUILD`	complete or place machines, storage, conveyors, or an EatingZone when material and a valid site exist
`WORK`	operate a feasible machine or complete output hauling to Exit-side storage
`EAT`	consume processed or raw food from inventory or nearby storage
`REST`	recover rest at a scored place

[19] See `SkillsComponent` in `src/components.h`, utility skill multipliers in `src/sim_utility.cpp`, execution XP in `src/sim_execute.cpp`, and `test_artistic_and_social_skills_progress`.

Action	Current effect or role
SOCIALIZE	interact with a nearby inhabitant, update social evidence and opinions, and possibly share food
CREATE	reduce expression and accumulate a discrete creative work unit at a scored walkable place
EXPLORE	choose a visible destination, reduce purpose, and possibly discover a hidden place
GET_FOOD	fetch processed food and available raw material from visible storage
MAINTAIN	repair a degraded adjacent conveyor
DISMANTLE	remove an eligible dead-end or blocking adjacent conveyor for a partial refund
SABOTAGE	under sufficient stress, damage adjacent built infrastructure
IDLE	explicit always-feasible no-effect alternative

CREATE is not restricted to `OpenSpace`, and REST or SOCIALIZE is not restricted to `EatingZone`. Cultural and restorative actions use scored places as described in section 15.

14.6 Decision and Movement Pipeline

Implemented. A decision proceeds through four distinct stages:

1. `system_compute_utility()` scores all thirteen actions and records `UtilityBreakdown {self, factory, cost, risk, final, feasible}`. Cost and risk are currently zero where no expected term is modeled; the structure does not imply that such terms already affect choice.
2. Feasible actions with positive score enter Boltzmann selection at canonical temperature 0.4. `IDLE` contributes score 0.02; utility-zero or infeasible actions have zero probability.
3. `system_find_targets()` chooses a target consistent with the same feasibility contracts. Invalidated plans fall back to `IDLE` and are measured separately from target failures.
4. `system_move_to_targets()` follows a cached cardinal A* route; execution changes state only after the target and spatial preconditions are reached.

Actions remain sticky for action- and personality-dependent durations, but may be released when no longer feasible or interrupted by survival need. Metrics

separate selection, target lookup, target arrival, and effective execution.

Implemented, with a locality limitation. Ordinary physical and social plans use Manhattan observation radius twelve, and a distant-stock counterfactual leaves utility, action, and target unchanged. A* may use the complete map after a target is selected. Conveyor route planning is the explicit exception: its helper uses factory-wide connectivity before the caller enforces a local returned site. The architecture is therefore mostly local, not fully local.[20]

14.7 Stress, Trauma, and Death

Implemented. Direct canonical stress input is added when hunger or rest exceeds 0.7 and when disease exceeds 0.3, then reduced by effective resilience. Passive recovery is

$$d_t = d_0 \left[1 + 8(1 - h_t)(1 - r_t)\right], \qquad d_0 = 0.005, \tag{37}$$

so recovery is faster when hunger and rest are satisfied. SOCIALIZE, CREATE, and EXPLORE can also reduce stress. Stress above 0.5 accumulates persistent trauma at `0.001` per tick; trauma reduces effective resilience and gregariousness.

Canonical `stress_model_variant = 1` derives continuous modifiers from the stress value: social approach falls, creativity/exploration can rise in a middle band, work is suppressed near maximum stress, and sabotage becomes possible above 0.6. `StressState` labels (`NORMAL`, `DISSOCIATED`, `HOSTILE_EUPHORIA`, `BROKEN`) are display and Chronicle categories in this variant, not a discrete behavioral state machine.

Death checks are exclusive within a tick. Starvation requires hunger at 1 for 180 consecutive ticks; exhaustion requires rest at 1 for 200. Above the canonical breakdown threshold 0.92, death is probabilistic rather than immediate, with a per-tick chance shaped from approximately 0.005 toward 0.003. Sabotage has a separate configured suicide chance of 0.03 per effective sabotage tick. Natural mortality uses lifecycle age. Canonical supply failure never directly assigns a factory-collapse death; it acts indirectly through material replenishment and physical needs.[21]

14.8 Lifecycle and Generations

Implemented. IDs are unique, monotonic, and never reused; dead entities remain available for history while releasing physical claims. Founders enter

[20] See `src/sim_utility.cpp`, `src/sim_targets.cpp`, `src/sim_movement.cpp`, and `src/grid.h`. The metrics contract requires thirteen action entries and zero target failures in its deterministic fixture.

[21] See stress and death systems in `src/simulation.cpp` and sabotage execution in `src/sim_execute.cpp`. Death exclusivity, suicide, and natural mortality have focused tests in `tests/simulation_tests.cpp`.

with ages 800–2000 ticks. Lifespan is a deterministic seed-and-ID value around 8000 ticks with 20% spread, and maturity begins at age 1200.

Arrivals are exogenous attempts at expected rate 0.8 per 1000 ticks. They enter through `Entrance`, do not inspect deaths or target population, start without relationships, and are discarded rather than queued when the living population has reached `max_population = 200`.

Reproduction is evaluated every 50 ticks for nearby mature pairs. Its continuous probability combines local food security, hunger, rest, stress, mood, reciprocal familiarity and trust, age, and a 1500-tick cooldown. Descendants inherit only mutated personality traits and genealogy. They do not inherit skills, XP, community labels, archetypes, relationships, opinions, place memory, inventory, or creative progress. Population can grow, decline, or become extinct without automatic replacement toward the initial 48 inhabitants.[22]

Established result. Deterministic 10,000-tick tests exercise historical IDs beyond simultaneous capacity, multiple cohorts, natural mortality, closed population accounting, and same-build replay. The Phase 7 five-seed CALM run also reached generations 2–3 with no founders surviving at tick 10,000.

Not established. Generational turnover does not by itself establish cultural inheritance, occupational dynasties, leadership succession, or subculture. The implemented inheritance rule deliberately does not copy those labels or states.

15 Emergent Spaces and Place Affinity

This section uses "emergent space" in a restricted, measurable sense: repeated spatial association that is not imposed by an exclusive action zone. It does not use the term as evidence of segregation, territory, subculture, free-riding, or leadership.

15.1 Physical Substrate and Affordances

Implemented. The 60×40 generated layout contains walls, ordinary floor, OpenSpace clusters, a central inherited EatingZone, renewable resource deposits, an Entrance, an Exit, and the inherited production chain. These stamps are designed physical conditions, not emergent zones. The presence of a central plaza or eating area therefore cannot itself be counted as self-organization.

Tile types constrain production actions: GATHER requires a source, WORK a machine or output destination, and maintenance/dismantling/sabotage nearby infrastructure. In contrast, REST, SOCIALIZE, and CREATE do not receive a categorical bonus from EatingZone or OpenSpace. CREATE is effective

[22] See `src/sim_lifecycle.cpp`, `[lifecycle]` in `config/default.toml`, and the lifecycle, inheritance, exogenous-arrival, and 10,000-tick tests in `tests/simulation_tests.cpp`.

on ordinary Floor, and EatingZone provides no canonical utility, satisfaction, movement-capacity, or community privilege.[23]

The human Director can set anonymous occupancy capacity on Floor or OpenSpace, place or remove infrastructure, and change maintenance priority. These are designed interventions and must be recorded separately from learned place use.

15.2 Personal Place Memory

Implemented. Every inhabitant stores at most 24 `PlaceMemoryEntry` records. An entry contains coordinate, affinity in $[-1, 1]$, exposure count, and last tick. After action execution, the current place receives an outcome based on the inhabitant's stress, excessive crowding, and whether an action had an effect. Effective eating, resting, socializing, and creating contribute positive evidence; effective work and building contribute a smaller positive value; sabotage and high stress contribute negative evidence. Repeated exposures update affinity toward the observed outcome, while the least recently used entry is replaced when memory is full.[24]

This is individual episodic affinity, not an objective tile quality and not a shared cultural label. Two inhabitants may evaluate the same coordinate differently because their histories differ.

15.3 Scored Places

Implemented. `find_preferred_place()` plans REST, SOCIALIZE, and CREATE from a candidate set containing the current tile, a stride-two sample of walkable tiles within Manhattan radius twelve, visible inhabitants, visible artifacts, and remembered places. Scores combine action-specific terms:

Action	Positive observations	Negative observations
REST	learned affinity, limited food access, personally valued artifacts	traffic, machinery noise, conveyor hazard, travel
SOCIALIZE	affinity, familiar/trusted people, moderate local presence	excessive traffic, noise, hazard, travel

[23] Layout generation is in `src/wfc_generator.h`; tile validity and walkability are in `src/components.h` and `src/grid.h`. `test_create_completes_discrete_work_units_on_ordinary_floor` and the static OpenSpace audit in `tests/verify_policy_audit.cmake` protect the nonexclusive cultural-action boundary.

[24] See `PlaceMemoryComponent` in `src/components.h`, `Simulation::system_spatial_learning()` in `src/simulation.cpp`, and `Simulation::find_preferred_place()` in `src/sim_targets.cpp`.

Action	Positive observations	Negative observations
CREATE	affinity, known people, personally valued artifacts	noise, hazard, excessive traffic, travel

The table describes implemented score terms, not guaranteed destinations. Place score only slightly modulates action utility; need, personality, stress, skill, and stochastic Boltzmann choice still matter.

Locality boundary. Current conditions are read only within observation radius twelve. A remembered place outside that radius may remain a candidate, but its score uses memory and travel cost rather than querying remote current traffic, people, food, hazard, or artifacts. Once selected, cached A* can inspect the full grid to route around walls. This distinguishes remembered destination choice from omniscient observation. Conveyor construction remains the separate global planning limitation described in section 3 and section 14.

15.4 Social Proximity Without a Privileged Zone

Implemented. Copresence within two tiles increases reciprocal familiarity but does not create trust. Effective shared BUILD, WORK, or CREATE creates reciprocal collaboration evidence. SOCIALIZE chooses among people within execution radius using familiarity, nonnegative trust, and distance, then may update opinions, stress, social skill, and food sharing. Eating near at least two others can satisfy a small amount of social need at any place. These mechanisms can make repeatedly used locations socially consequential without declaring them a home, faction, or subculture.

Graph-community labels are derived from reciprocal trust and familiarity every fifty ticks, but behavior is prohibited from reading those labels. Spatial and social persistence are therefore observations over continuous relations, not bonuses attached to a named group.

15.5 Measured Persistence Result

Established, limited result. Phase 6 recorded a paired CALM comparison over 20 seeds at 1000 ticks. All variants retained 48/48 inhabitants. Mean temporal Jaccard persistence of pairs within Manhattan radius three was:

Variant	Mean spatial persistence
Full mechanisms	0.238
Spatial affinity disabled	0.198

Pairs were sampled every fifty ticks. At this horizon and under this metric, enabling personal place affinity increased mean short-to-medium-term spatial

persistence by 0.040. This is evidence for a limited mechanism effect, not for stable districts or identity-based clustering.[25]

The same experiment reported a personality-distance difference against a deterministic trait shuffle, but it did not estimate confidence intervals or establish long-horizon persistence. It is therefore insufficient for a segregation claim.

15.6 Background and Unestablished Hypotheses

Background only. Schelling's model relocates agents according to the fraction of similar neighbors and a tolerance threshold (Schelling 1971). The present model has neither mechanism: it has no binary similarity class, no tolerance test, and no relocation rule triggered by unlike neighbors. Place affinity must not be described as a formal Schelling analogue.

Not established. Current code and experiments do not establish:

- segregation by personality, opinion, origin, or any other identity;
- an artistic or occupational subculture;
- territorial ownership or exclusion;
- leadership caused by spatial precedence or centrality;
- free-riding or a spatial cooperation equilibrium.

Testing any of these would require a phenomenon-specific metric, longer runs, multiple seeds, uncertainty estimates, and a causal counterfactual. For example, free-riding would require a contribution-benefit definition and evidence that low contribution systematically yields disproportionate benefit; the recorded Phase 6 contribution-benefit correlation was positive, not evidence for that claim.

16 Social Fabric: Evidence, Directed Relations, and Claims

The social layer is implemented, but not every collective pattern that it can support has been demonstrated. This section therefore separates three levels: the state and update rules executed by the program, observational summaries derived from that state, and emergence hypotheses that still require a multiseed counterfactual.

[25] The metric implementation is `Simulation::system_record_emergence_metrics()` and its JSON fields are defined in `src/metrics.h` and `src/batch_main.cpp`. The 20-seed paired result is recorded in the Phase 6 result of `doc/plans/2026-07-21-alineacion-diseno-implementacion.md`; deterministic metric accumulation and toggles are checked by `test_metrics_contract` and `tests/verify_metrics.cmake`.

16.1 Implemented State: a Directed Relationship Graph

`SocialFabric` stores a dynamically expandable flat matrix of `RelationshipEntry` values (`src/social.h`). For two historical agent IDs i and j, the entry

$$R_{ij} = (f_{ij}, t_{ij}, \ell_{ij})$$

is **agent i's relation toward agent j**. Familiarity $f_{ij} \in [0, 1]$, trust $t_{ij} \in [-1, 1]$, and the last-evidence tick ℓ_{ij} are directional: in general, $R_{ij} \neq R_{ji}$. The matrix grows when monotonic historical IDs exceed its current capacity, so dead agents and new cohorts do not require ID reuse (`SocialFabric::ensure_agent_id`).

All evidence updates use the common rule

$$f' = \text{clamp}_{[0,1]}\left(f + g_f(1 - f)\right), \qquad t' = \text{clamp}_{[-1,1]}(t + g_t).$$

Trust is therefore an additive evidence balance; it is not multiplied by familiarity during reinforcement. With social learning enabled, the active evidence sources are:

Evidence	Direction updated	g_f	g_t	Runtime source
Copresence within Manhattan distance 2	both $i \to j$ and $j \to i$	0.001	0	`system_social_learning()`
Effective shared BUILD, WORK, or CREATE within distance 2	both directions	0.003	0.002	`system_social_learning()`
Explicit SOCIALIZE interaction with the selected neighbor	both directions	0.05	0.03	`system_execute_actions()`
Help of amount a from i to j	recipient $j \to i$	$0.01a$	$0.03a$	`record_help()`

Evidence	Direction updated	g_f	g_t	Runtime source
The helper's observation of the recipient	helper $i \to j$	$0.005a$	0	`record_help()`
Negative observation of actor j by observer i	observer $i \to j$ only	0.01	$-s$	`record_negative_observation()`

The collaboration row requires both agents to report an effective action in the current tick; sharing an action label without an effect does not create trust. The directional help rule is equally important: receiving food is evidence for trusting the helper, but helping does not grant the helper reciprocal trust by fiat. Negative evidence similarly changes only the observer's edge toward the observed actor. Resident-to-resident execution rules generate such local evidence for witnessed sabotage and, when an observer's work-ethic opinion exceeds 0.65, eating near a machine; the canonical institution receives no report. An unrepaired dismantle site also creates directed negative evidence (section 20). Watcher reports to the institution are retained only in the noncanonical `external.policy_variant = 0` branch (`src/sim_execute.cpp`).

After 100 ticks without evidence, familiarity loses 0.0001 per tick. Trust moves 0.1% of its current distance toward zero every tick. These two decay rules run after contagion, influence, and mood updates in `Simulation::advance()`.

16.2 Social Action, Help, and Collaboration

SOCIALIZE is selected by the ordinary utility process. At execution it searches agents within Manhattan distance 6 and chooses by the acting agent's directed familiarity and positive trust, with distance and ID tie-breaking. Its social-need satisfaction depends on gregariousness, social skill, and crowd size: two nearby agents multiply satisfaction by 1.5 and three or more by 2.0. Trust does **not** directly multiply this satisfaction. Instead, positive t_{ij} reduces the actor's stress, affects opinion learning, and contributes to willingness to share surplus food (`src/sim_execute.cpp`). Crowds of at least three also provide a small passive social-need reduction even when the agent is doing another action.

When an agent works near a trusted acquaintance, `collaboration_bonus()` reads the actor's edge R_{ij}. For agents within distance 2 satisfying $f_{ij} \geq 0.1$ and $t_{ij} \geq 0.1$, the increment is

$$b_{ij} = 0.2 \max(0, t_{ij})(0.5 + 0.5 f_{ij}),$$

and the total multiplier is $1 + \min(1, \sum_j b_{ij})$. The result is in $[1, 2]$ and is applied to machine WORK. BUILD has an additional co-builder multiplier and directed-trust adjustment in `src/sim_execute.cpp`; these are mechanistic productivity effects, not proof that stable teams or occupations have emerged.

16.3 Contagion, Grief, and Influence

16.3.1 Stress contagion

For each index pair $i < j$ in the current alive-vector iteration, `apply_contagion()` reads only the directed edge from the first indexed agent to the second. If that edge has $f_{ij} \geq 0.05$, it computes

$$q_{ij} = 0.02 \, |t_{ij}| (0.3 + 0.7 f_{ij})(\sigma_i - \sigma_j)(1 - r_j),$$

then applies $\Delta\sigma_i = -q_{ij}$ and $\Delta\sigma_j = +q_{ij}$, clamped to $[0, 1]$. Antagonistic and positive ties both transmit stress because the rule uses $|t_{ij}|$. This is the exact implementation, including its asymmetry: it does not average R_{ij} with R_{ji}, and the susceptibility factor is always that of the second member in the iterated pair. It should not be described as a general symmetric diffusion operator.

16.3.2 Grief

Every terminal cause uses `Simulation::kill_agent()`, which queues the deceased ID once. During the death system, each survivor reads its own edge toward the deceased, R_{sd}. For $f_{sd} \geq 0.1$ the stress increment is

$$\Delta\sigma_s = 0.05 f_{sd} \max(0, t_{sd})(1 - r_s).$$

The survivor's familiarity with the deceased is then multiplied by 0.8. This implements a grief impulse and can provide input to later contagion. It does not, by itself, establish that a multistep grief cascade or secondary mortality occurs at population scale (`src/social.h`, `src/simulation.cpp`).

16.3.3 Influence is a score, not a discovered leader

`SocialComponent::influence` is a smoothed continuous score. For agent i, the averages are calculated from **incoming** edges R_{ji} for living j with $f_{ji} > 0.05$: influence represents being known and trusted by others. The target is

$$I_i^* = c_i (1 - \sigma_i)(0.3 + 0.7 \bar{f}_{\to i})(0.5 + 0.5 \max(0, \bar{t}_{\to i})),$$

and $I_i \leftarrow I_i + 0.05(I_i^* - I_i)$. Influence affects SOCIALIZE utility, food-sharing willingness, and DeGroot weights during bounded-confidence opinion exchange. It is not betweenness, eigenvector centrality, an assigned office, or a behavioral group label. Calling a high-I agent a leader remains an interpretation until temporal precedence and effects are tested.

16.4 Observational Communities and Metrics

Every 50 ticks, `system_community_detection()` derives connected components from reciprocal evidence: both directed edges must have trust above 0.3 and familiarity above 0.2. Components smaller than three remain unlabeled. The resulting `AgentComponent::community_id` is rebuilt for observation and Chronicle events; static tests prohibit its use in utility, targeting, execution, and institutional policy (`tests/verify_policy_audit.cmake`).

The schema-3 `vida_batch metrics` record reports directed relationship-edge counts, average directed trust/familiarity, graph modularity, community-pair stability, spatial-pair persistence, personality-distance comparison, action entropy, specialization summaries, and a contribution/food-benefit ledger (`src/metrics.h`, `src/batch_main.cpp`). These are measurements, not automatic classifiers of culture.

16.5 Demonstrated Results and Open Hypotheses

The Phase 6 paired CALM experiment used 20 seeds and independent mechanism toggles. At 1000 ticks, disabling social learning reduced measured modularity and community stability to zero; disabling spatial affinity reduced spatial-pair persistence relative to the full model. Those results support the narrower claims that relationship evidence creates the measured graph components and learned place affinity contributes to short-horizon spatial persistence (`doc/plans/2026-07-21-alineacion-diseno-implementacion.md`).

The same experiment did **not** establish the following stronger claims:

- **Spatial segregation:** the personality-distance delta lacked uncertainty estimates and long-horizon persistence.
- **Artistic subcultures:** the model has artistry but no shared aesthetic preference variable, so this hypothesis is not operationalized.
- **Informal leadership:** influence exists and has downstream coefficients, but causal leader formation was not tested.
- **Free-riding:** the observed contribution-benefit correlation was positive; exploitation by a noncontributing subgroup was not demonstrated.
- **Collective slowdown or strike:** no threshold transition or coordinated work withdrawal has been isolated from needs, logistics, and factory pressure.
- **Grief cascades:** grief and contagion are active mechanisms, but a cascade of secondary outcomes has not been shown by a multiseed counterfactual.

- **Generational culture:** turnover is implemented, but persistence or divergence of norms across cohorts has not yet been identified (section 17).

Accordingly, specialization, leadership, subculture, segregation, free-riding, slowdown, and generational continuity remain hypotheses unless a metric, counterfactual, horizon, and multiseed result are stated with them.

16.6 Implementation and Verification References

- State and update rules: `src/social.h`, `src/simulation.cpp`, `src/sim_execute.cpp`.
- Utility and place effects: `src/sim_utility.cpp`, `src/sim_targets.cpp`.
- Configuration: `[culture]` and canonical `[external].policy_variant = 1` in `config/default.toml`; fields in `src/config.h` and parsing in `src/config.cpp`.
- Directionality and label neutrality tests: `test_social_evidence_is_directional()` and `test_graph_labels_are_behavior_neutral()` in `tests/simulation_tests.cpp`, plus `tests/verify_policy_audit.cmake`.
- Metrics contract and counterfactual toggles: `tests/verify_metrics.cmake`.

17 Life, Death, Arrivals, and Generations

The executable has an active demographic system. It combines material and psychological mortality, age-dependent natural death, an exogenous arrival process, and endogenous reproduction. None of these mechanisms restores the population to `initial_population`: the number alive may rise to `max_population`, fall below the founding cohort, or remain at zero.

17.1 Identity and Lifecycle State

Every person receives a monotonic historical ID and an ECS entity that is never reused. `LifecycleComponent` records (`src/components.h`):

Field	Meaning
`origin`	initial founder, exogenous arrival, or birth
`parent_a`, `parent_b`	biological parent IDs for births; -1 otherwise
`entry_tick`, `age_at_entry`	when the person entered and its age then
`lifespan`	deterministic individual natural-death age
`cohort`	temporal entry cohort
`generation`	genealogical generation
`last_reproduction_tick`	pair cooldown state

Field	Meaning
`death_tick`	terminal tick, or -1 while alive
`first_trusted_edge_tick`	first reciprocal familiarity/trust integration event
`peak_influence`	largest observed continuous influence score

Age is derived rather than incremented:

$$a_i(t) = a_i^{\text{entry}} + \max(0, t - t_i^{\text{entry}}).$$

The temporal cohort is $\lfloor t_i^{\text{entry}} / w_c \rfloor$, with default width $w_c = 1000$ ticks. Cohort and generation are deliberately different: unrelated arrivals in the same time window share a cohort, while a child's generation is one plus the larger parental generation.

`SocialFabric`, Chronicle, and per-agent metric ledgers expand as historical IDs grow. Dead entities remain in the registry for genealogy and post-hoc analysis, but all behavior systems skip `alive == false`.

17.2 Founding Population

`spawn_initial_agents()` creates the configured 48 founders on independently sampled walkable Floor or OpenSpace positions (`config/default.toml`). A founder first samples one of six archetypes uniformly, then jitters its six continuous traits around that archetype's values and clamps them to $[0.05, 0.95]$. The archetype is a display provenance for founders, not a profession assignment.

Founding needs are sampled independently from $[0, 0.25]$; opinion priors come from the sampled archetype plus ± 0.10 noise; skills and XP start at zero; stress starts at zero; place memory and relationships are empty. Each founder receives the configured bootstrap reserve of 5 processed-food units. Founder age is a deterministic seed/ID hash in the configured interval $[800, 2000]$ ticks.

For every origin, lifespan is another seed/ID hash:

$$L_i = \text{round}\left(L_0[1 + s(2u_i - 1)]\right),$$

where default life expectancy $L_0 = 8000$, spread $s = 0.20$, and $u_i \in [0, 1)$. The implementation also enforces $L_i \geq a_{\text{maturity}} + 1$ (`src/sim_lifecycle.cpp`).

17.3 Mortality and the Exclusive Death Pipeline

Needs decay at the beginning of a tick, before decisions and actions. Stress and death checks occur much later, after production, policy, artifact, and community

systems (section 18). `system_check_deaths()` evaluates the following causes in strict precedence order:

Cause	Active trigger under the default configuration
Starvation	hunger remains exactly at 1 for 180 consecutive death checks
Exhaustion	rest remains exactly at 1 for 200 consecutive death checks
Breakdown	stress is at least 0.92, followed by a probabilistic death draw
Natural	derived age reaches the individual's lifespan

The hunger and rest counters reset to zero as soon as the associated need falls below 1. In the canonical continuous stress model, breakdown is **not immediate**: the per-tick probability is 0.005 at the threshold and smoothly decreases to 0.003 as stress approaches 1. The lower probability at the most extreme stress is current executable behavior, intended to leave time for sabotage or recovery; it should not be documented as a deterministic ceiling death (`src/simulation.cpp`, `[stress]` and `[death]` in `config/default.toml`).

Natural mortality is checked only after the three preceding causes. Thus, if starvation and lifespan coincide, the recorded cause is starvation. Suicide can occur during SABOTAGE execution and enters the same pipeline through `kill_agent()` before the later death check. `DeathCause::COLLAPSE` remains a typed historical category, but canonical factory collapse does not directly kill agents.

`kill_agent()` is the sole terminal operation. It sets the alive flag and typed cause, records `death_tick`, releases all tile claims held by that ID, emits one factual Chronicle event, increments suicide state when applicable, and queues one generic grief application. `record_metric_deaths()` subsequently records each ID once. Factory health and quota never call a direct population-kill operation.

17.4 Exogenous Arrivals

Arrivals are active by default and are not death replacements. At the end of each tick, `system_lifecycle()` evaluates a deterministic Bernoulli event with

$$p_A = 1 - \exp(-\lambda_A/1000),$$

where the default rate is $\lambda_A = 0.8$ attempts per 1000 ticks. The draw is a hash of seed and tick; it does not inspect deaths, `initial_population`, factory state,

relationships, or the shared behavioral RNG. A death can only affect whether capacity is available when an already scheduled attempt occurs.

If the number alive has reached `max_population = 200`, the attempt is counted as blocked and discarded. It is not queued. Otherwise, the new person appears at the first Entrance tile and receives:

- six independent traits in $[0.1, 0.9]$ and no assigned archetype;
- four neutral opinions in $[0.45, 0.55]$;
- hunger, rest, social, expression, and purpose independently in $[0, 0.2]$;
- an empty inventory, no skills or XP, no relationships, no place memory, and zero stress;
- a deterministic entry age in the default interval $[1200, 3000]$.

The arrival is recorded with origin `ARRIVAL`, no parents, a new monotonic ID, and the temporal cohort of its entry tick. Because lifecycle runs after all behavior and metrics for the tick, the newcomer first acts on the following tick.

17.5 Endogenous Reproduction

Reproduction is also active by default. It is evaluated every 50 ticks at a base rate of 1.0 per 1000 ticks. Candidate agents are sorted by historical ID. A pair is eligible only when both agents:

- are at least 1200 ticks old;
- have completed the 1500-tick reproduction cooldown;
- are within Manhattan distance 3;
- have positive reciprocal familiarity and trust evidence.

The social factor is

$$S_{ab} = \sqrt{f_{ab} f_{ba} \max(0, t_{ab}) \max(0, t_{ba})}.$$

This is multiplied by continuous material, wellbeing, and age factors. Material state combines each parent's hunger/rest satisfaction with local food security: personal food plus 10% of food and raw-food stocks in Storage within Manhattan radius 12, normalized and clamped. Wellbeing combines both stress levels and moods. The age factor ramps up after maturity and falls between 75% and 95% of individual lifespan. For combined score Q_{ab} and check interval $\Delta = 50$, the birth probability is

$$p_B = 1 - \exp\left(-\lambda_B \Delta Q_{ab} / 1000\right).$$

The draw is deterministic from seed, reproduction epoch, and parent IDs. Material abundance alone cannot bypass absent reciprocal social evidence. If a successful draw occurs at live capacity, the blocked birth is counted and discarded; there is no deferred pregnancy or replacement queue.

17.6 Inheritance and Genealogy

A child appears at the first parent's current position with age zero, origin `BIRTH`, both parent IDs, and generation $1+\max(g_a, g_b)$. For each personality dimension k,

$$x_k^{\text{child}} = \text{clamp}_{[0.05,0.95]}\left(\frac{x_k^a + x_k^b}{2} + \epsilon_k\right), \qquad \epsilon_k \in [-0.08, 0.08].$$

The child starts with each ordinary need at 0.05, opinions at 0.5, empty inventory, zero skills/XP, no archetype, no relationships, no community label, no place memory, and no inherited creative progress. Genealogy therefore records descent but does not copy a role, social group, learned relation, spatial culture, or practiced skill. Subsequent cultural similarity must be produced by later interaction, not by a hidden descendant label.

17.7 Integration, Cohorts, and Metrics

At each lifecycle pass, an agent is marked socially integrated for metrics when there exists another living agent such that both directed edges have familiarity and trust at least 0.1. `first_trusted_edge_tick - entry_tick` is reported as integration latency. Peak influence is updated continuously. These quantities are descriptive mobility proxies, not proof of assimilation or leadership.

Schema-3 metrics (`src/batch_main.cpp`) close historical population accounting:

$$N_{\text{ever}} = N_{\text{initial}} + N_{\text{arrivals}} + N_{\text{births}} = N_{\text{alive}} + N_{\text{deaths}}.$$

They report attempts, admissions, capacity blocks, origins among survivors, deaths by cause, temporal cohorts with person-ticks and live censoring, and a per-person genealogy containing age, lifespan, parents, generation, integration, influence, and traits. The field named `integrated_descendants` counts all integrated non-founders, including both arrivals and births.

17.8 What Has and Has Not Been Demonstrated

The Phase 7 verification ran deterministic 10,000-tick CALM simulations. Across the established seeds, no founder remained alive; multiple cohorts, arrivals, births, natural deaths, and generations were observed, and population peaks could rise above 48 before falling well below it. Separate fixtures demonstrate persistent extinction when arrivals and reproduction are disabled, and identical arrival schedules in worlds differing only by an unrelated death. These are evidence for demographic turnover and for the absence of target-population replacement (`tests/simulation_tests.cpp`).

They do **not** demonstrate preservation, loss, or divergence of a culture across generations. The executable records the genealogy and social/spatial observations needed for such a study, but no multiseed intergenerational norm metric or counterfactual has yet established institutional memory, generational conflict, or inherited subculture. Those outcomes remain hypotheses.

17.9 Implementation and Verification References

- Lifecycle implementation: `src/sim_lifecycle.cpp`; spawning and death pipeline: `src/simulation.cpp`; state types: `src/components.h`.
- Active defaults: `[simulation]`, `[lifecycle]`, `[death]`, and `[stress]` in `config/default.toml`; declarations and parsing in `src/config.h` and `src/config.cpp`.
- Focused tests: `test_natural_mortality_uses_exclusive_death_pipeline()`, `test_arrivals_are_exogenous_and_newcomers_start_empty()`, `test_reproduction_inherits_traits_not_roles_or_relationships()`, and `test_dynamic_identity_and_ten_thousand_tick_turnover()` in `tests/simulation_tests.cpp`.
- Schema and accounting checks: `tests/verify_metrics.cmake`; static prohibition of inherited behavioral labels: `tests/verify_policy_audit.cmake`.

18 Runtime Architecture, Tick Pipeline, and Interfaces

The current program is a hybrid simulation architecture, not a pure ECS and not a terminal prototype. EnTT manages dynamic entities and their components, while a dense grid, a directed relationship matrix, event stores, configuration, metrics, and institutional state are owned directly by `Simulation`.

18.1 Hybrid Runtime Model

The main state holders are:

Subsystem	Representation	Primary references
Agents and artifacts	EnTT entities with plain data components	`src/components.h`, `Simulation::registry_`
Physical map	Dense `Grid` arrays of `TileType` and `TileData`	`src/grid.h`
Directed relationships	Dynamically expandable flat matrix indexed by historical IDs	`src/social.h`
Production assessment	Value snapshot recomputed after transport and shipping	`src/production.h`

Subsystem	Representation	Primary references
Chronicle	Typed factual event collection and narrative rendering	`src/chronicle.h`
Metrics	Structured counters, sums, and per-agent ledgers	`src/metrics.h`
Human interventions	Typed command variant and ordered event ledger	`src/director.h`
Orchestration	Stateful `Simulation` object and member systems	`src/simulation.h`

EnTT components are data-oriented, but the systems are not independent stateless functions: they are `Simulation` member functions and communicate through the registry, `Grid`, `SocialFabric`, metrics, Chronicle, RNG streams, and aggregate state. Tile entities are not stored in EnTT. This distinction matters when reasoning about dependencies and tests.

The project requires C++20. CMake fetches EnTT v3.13.2 and tomlplusplus v3.4.0 from pinned Git tags. SDL is isolated behind the GUI build option (`CMakeLists.txt`).

18.2 Executable Targets

The build defines these targets:

Target	Availability	Purpose
`vida_gui`	when `VIDA_BUILD_GUI=ON`	SDL2/SDL2_ttf interactive 2.5D isometric client
`vida_batch`	always	headless commands, experiments, metrics, Chronicle export, and replay
`vida_tests`	when `BUILD_TESTING=ON`	deterministic C++ simulation tests

There is no current `vida_misma` TUI executable. Both user-facing executables load `config/default.toml` (or `../config/default.toml`) once, construct the same `Simulation`, and use the same source list. The batch CLI provides `run`, `calm`, `production`, `culture`, `story`, `agent`, `analysis`, `jsonl`, `map`, `metrics`, and `replay` commands (`src/batch_main.cpp`).

18.3 Source Decomposition

`VIDA_SIM_SOURCES` in `CMakeLists.txt` is the shared executable core:

Source	Responsibility
`src/simulation.cpp`	construction, tick orchestration, resource regeneration, shipping, stress/death, social observations, aggregate metrics, and historical legacy pressure
`src/sim_utility.cpp`	local observations, utility decomposition, feasibility, and Boltzmann action selection
`src/sim_targets.cpp`	action-specific target plans and physical claims
`src/sim_movement.cpp`	occupancy-limited movement over cached A* paths
`src/sim_execute.cpp`	action effects, production transformations, resource transfer, help, construction, maintenance, dismantling, and sabotage
`src/sim_conveyor.cpp`	condition decay and one-resource conveyor transfer
`src/sim_lifecycle.cpp`	spawning primitive, age, arrivals, reproduction, genealogy, and integration observations
`src/sim_policy.cpp`	canonical anonymous physical institutional policy
`src/sim_space_policy.cpp`	anonymous occupancy-capacity policy
`src/director.cpp`	intervention-log TOML schema and parsing/serialization
`src/sim_director.cpp`	validation and application of typed Director commands

Header-only services include `Grid`, A* and path caching, production assessment, social rules, recipes, components, metrics, and Chronicle. `src/config.cpp` is compiled separately into both user-facing executables. GUI-only sources are `src/main_gui.cpp`, `src/graphical_view.cpp`, and `src/font_cache.cpp`.

18.4 Exact `Simulation::advance()` Pipeline

The executable order is the order below (`src/simulation.cpp`). It is not the seven- or eight-stage loop described by earlier versions of this document.

1. `system_regen_resources()`
2. `system_decay_needs()`

```
 3. apply CALM state, or grow quota when it was not manually fixed
 4. system_compute_utility()
 5. system_find_targets()
 6. system_move_to_targets()
 7. system_execute_actions()
 8. system_social_learning()
 9. system_spatial_learning()
10. system_conveyor_transport()
11. system_ship_out_food()              # output shipping despite legacy name
12. ProductionChain::assess()            # snapshot for the next tick
13. non-CALM institutional policy:
      supply variant 0: factory deterioration
      policy variant 0: legacy restructure + hidden-space exposure
      policy variant 1: indifferent restructure + space overcapacity
14. supply variant 1, non-CALM: system_update_factory_condition()
15. system_artifact_effects()
16. system_community_detection()
17. system_update_stress()
18. system_check_deaths()                # includes pending grief
19. SocialFabric: contagion, influence, mood, relationship decay
20. system_check_dismantle_penalties()
21. system_record_emergence_metrics()
22. system_chronicle_narrative()
23. record_metric_deaths()
24. system_lifecycle()                   # integration, arrivals, reproduction
25. metrics_.ticks_advanced++; tick_++
```

Several timing consequences follow:

- Need decay precedes action, so an action observes already-decayed needs and may satisfy them later in the same tick.
- Social and spatial evidence observe action effects after execution.
- Conveyors transport resources deposited by WORK in the same tick; agents can use the resulting Storage stock no earlier than the next action phase.
- Shipping drains only output that has physically reached qualifying Storage by stage 11. The support signal then affects regeneration first on the next tick.
- `ProductionChain::assess()` is post-action and post-shipping. Its stored snapshot is exposed for diagnostics; canonical utility and targeting do not consume it and mostly use local observations. Conveyor connectivity planning is a declared global scan, and A* reads the full map only after target selection.
- Death occurs before graph contagion but grief is applied inside the death stage; dead agents are absent from the later alive-vector social updates.
- Death metrics are recorded before lifecycle creates arrivals and births. Newcomers therefore begin acting on the following tick.

Director interventions are intentionally outside this function. Their event phase is `before_advance`: GUI commands and replay events are applied at the current `Simulation::tick()` before calling `advance()`.

18.5 SDL Graphical Client

`vida_gui` is an SDL2/SDL2_ttf renderer with an isometric tile view, camera pan and zoom, agent selection/following, speed control, pause, single-step, factual event display, tile tooltips, and agent panels (`src/graphical_view.h`, `src/graphical_view.cpp`). Simulation and rendering run on the same thread.

Normal view exposes environmental consequences such as quota/fill, stocks, shipping, condition, density, zones, maintenance priority, and events. `F12` explicitly enables debug information, including exact needs, personality, relations, and utility decomposition. `E` enters Director edit mode and pauses advancement. `vida_gui --seed N --record FILE` writes accepted interventions when the session exits (`src/main_gui.cpp`). SDL2 and SDL2_ttf are resolved through CMake package targets, including vcpkg, with a Unix/Homebrew library fallback.

18.6 Typed Director Boundary

`DirectorCommand` is a `std::variant` containing only:

- `DirectorSetQuota`;
- `DirectorSetZone` with anonymous occupancy capacity $0 \dots 8$;
- `DirectorPlaceStructure` for Wall, Storage, one of three Machines, or Conveyor;
- `DirectorRemoveStructure`;
- `DirectorSetMaintenancePriority` for a built conveyor.

The API accepts environmental coordinates and physical parameters, never agent identity, action, behavioral target, personality, relationship, community, or utility state (`src/director.h`, `src/sim_director.cpp`). Placed structures are completed immediately. Priority is a signal consumed by autonomous maintenance; it does not repair a belt or assign a worker. CALM rejects quota intervention but continues to permit physical editing.

Each accepted command receives the current tick and a strict sequence number. Rejected commands are atomic and do not enter the ledger. Intervention logs use TOML format `vida-interventions`, schema 2, `tick_phase = "before_advance"`, and a fingerprint of the loaded configuration source (`src/director.cpp`).

`vida_batch replay <ticks> <seed> <file>` validates format, mode, seed, configuration-source fingerprint, ordered sequence, event ticks, and each command transition. It emits a separate replay JSON schema 1 with a state fingerprint. The guarantee tested by the project is determinism within the

same build and configuration; the format is not a promise of bit-identical replay across compiler, platform, or future schema and model versions.

18.7 Schema-3 Metrics Contract

`vida_batch metrics` emits exactly one JSON object with `schema_version: 3`. Its top-level evidence includes:

Section	Content
`population`	initial/max/alive/peak/historical counts, arrivals, births, capacity blocks, deaths by cause
`demography`	mechanism toggles, integration summaries, temporal cohorts, and per-person genealogy/lifecycle records
`factory`	quota demand, output shipped/produced/hauled, support/supply factor, policy variants, intervention window, deterioration events
`actions`	selected, lookup, failed, invalidated, reached, executed, and utility decomposition for all 13 actions
`resources`, `machines`, `stocks`	physical flow accounting and inherited/current infrastructure
`social`, `emergence`	directed-edge aggregates, communities, persistence/modularity/stability, specialization, contribution/benefit, per-agent ledger
`needs`, `skills`, `events`, `timeline`	terminal aggregates, factual counters, and optional samples

The command accepts explicit toggles for social learning, spatial affinity, artifact effects, natural mortality, arrivals, and reproduction, enabling paired counterfactuals. Schema 3 is the metrics contract; it should not be confused with the schema-2 intervention log or schema-1 replay result (`src/batch_main.cpp`).

18.8 Verification and Phase Status

CMake registers four CTest entries when testing is enabled:

Test	Coverage
`vida.metrics`	C++ fixtures for simulation, map, logistics, social directionality, mortality, demographics, Director, and deterministic replay
`vida.cli_metrics`	byte-identical same-build schema-3 output, accounting, sections, toggles, and CLI validation
`vida.policy_audit`	static institutional, group-label, RNG, lifecycle-label, and Director-boundary prohibitions
`vida.cli_replay`	byte-identical same-build replay and rejection of a mismatched seed

The alignment plan's Phases 0 through 9 are complete. Phase 9 consolidated the documentation so claims in present tense point to executable mechanisms, while emergent outcomes require stated evidence (`doc/plans/2026-07-21-alineacion-diseno-implementacion.md`).

18.9 Implementation and Verification References

- Build graph and targets: `CMakeLists.txt`.
- Orchestration and shared state: `src/simulation.h`, `src/simulation.cpp`.
- Batch and schema-3 output: `src/batch_main.cpp`, `src/metrics.h`.
- GUI: `src/main_gui.cpp`, `src/graphical_view.h`, `src/graphical_view.cpp`.
- Director and replay: `src/director.h`, `src/director.cpp`, `src/sim_director.cpp`, `tests/replay_fixture.toml`, `tests/verify_replay.cmake`.
- Test contracts: `tests/simulation_tests.cpp`, `tests/verify_metrics.cmake`, `tests/verify_policy_audit.cmake`.

19 Production Pipelines and Physical Conveyor Logistics

The executable starts with a degraded, inherited three-machine factory. Agents can operate it without founding the production chain, then maintain, haul, and extend its logistics. Conveyors are autonomous one-resource buffers, but agents remain necessary for machine work, construction, maintenance, and fallback hauling.

19.1 Active Three-Stage Factory

The material transformations are:

$$\text{raw food} \xrightarrow{\text{FoodMachine}} \text{food},$$

$$\text{raw material} \xrightarrow{\text{MaterialsMachine}} \text{construction material},$$

$$\text{construction material} \xrightarrow{\text{OutputMachine}} \text{output} \xrightarrow{\text{Exit Storage}} \text{shipped quota}.$$

FoodMachine and MaterialsMachine are inherited on renewable resource deposits and auto-gather a small amount into their machine buffers. OutputMachine consumes construction material. Machine transformations still require an agent executing WORK; conveyors do not operate machines (`src/simulation.cpp`, `src/sim_execute.cpp`, `src/recipes.h`).

Before the founding population is spawned, WFC places one built machine of each type, at least three Storages, and four built eastbound conveyors joining the OutputMachine to Exit-adjacent Storage. Initial belt conditions are 0.55, 0.65, 0.75, and 0.85. Property tests verify this reachable minimum chain over 20 seeds, and a focused fixture produces all three outputs and ships without any BUILD action (`src/wfc_generator.h`, `test_inherited_factory_map_properties()` and `test_inherited_chain_operates_without_build()` in `tests/simulation_tests.cpp`).

19.2 Conveyor State and Walkability

Each conveyor uses the general structure fields `built`, `build_progress`, and `build_cost`, plus (`src/components.h`):

Field	Executable meaning
`conveyor_dir`	one of N, S, E, W; selects exactly one target tile
`conveyor_condition`	$[0, 1]$ mechanical condition
`conveyor_contents_type`	the single resource type currently represented
`conveyor_contents`	amount buffered on this segment
`maintenance_priority`	anonymous normal/high Director signal

Conveyors are walkable. `Grid::is_walkable()` rejects only Wall, so agents can stand on and path across built or unbuilt belts. A conveyor cannot be a path blocker in the current movement model, and

`Grid::is_conveyor_blocking_path()` therefore always returns false (`src/grid.h`, `src/pathfinding.h`).

MAINTAIN and DISMANTLE nevertheless use an adjacent-interaction plan: target selection sends the agent to a walkable neighbor and execution scans the eight-neighborhood for the belt. BUILD can complete an unbuilt conveyor frame from an adjacent Floor path or while standing on the walkable frame (`src/sim_targets.cpp`, `src/sim_execute.cpp`). There are no separate DEPOSIT or RETRIEVE actions in `ActionType`; machine output deposition and agent hauling are effects within WORK and the execution pass.

19.3 One-Resource Transport Semantics

A segment may contain only one resource type at a time. If its amount is nonzero, a deposit or upstream transfer of a different type is rejected. When empty, its type tag may be replaced. With configured throughput $\tau = 0.5$, the same value acts both as maximum transfer per tick and as the effective segment capacity:

$$m = \min(a_c, \tau), \qquad m_{c \to n} = \min(m, \max(0, \tau - a_n))$$

for a compatible downstream conveyor. The sender loses exactly the amount the receiver accepts. This one-type contract and partial-deposit conservation are covered by `test_conveyors_do_not_mix_resources()` and `test_partial_conveyor_deposit_preserves_remainder()` (`tests/simulation_tests.cpp`).

`system_conveyor_transport()` collects built belts and sorts them by increasing Manhattan distance to the nearest Exit. It then processes each belt once. This geometric downstream-first order prevents double movement on the inherited chain and on routes whose every step approaches Exit. It is not a topological sort of an arbitrary directed network; equal-distance, looping, or temporarily away-from-Exit layouts do not have a general one-segment-per-tick proof (`src/sim_conveyor.cpp`).

For a belt whose condition remains at least 0.2 after the current tick's decay, the target behavior is:

Target	Transfer rule
Built compatible Conveyor	move into free segment capacity; mismatched type, broken target, or full target backs up

Target	Transfer rule
Built Machine	accept only RAW_FOOD to FoodMachine, RAW_MATERIAL to MaterialsMachine, or CONSTRUCTION_MATERIAL to OutputMachine, up to a small input buffer
Storage not near Exit	accept non-output resources up to shared Storage capacity; OUTPUT remains on the belt
Storage within Manhattan radius 3 of Exit	accept every represented resource, including OUTPUT, up to capacity
Exit	try to deposit into a cardinally adjacent Storage; without one, the belt backs up
Floor, Wall, incompatible or unbuilt structure	no transfer

An Output belt aimed at non-Exit Storage does not pass through that Storage: it backs up on its current segment. The source comment saying that such output "continues past" contradicts the implemented branch; there is no pass-through operation. This remains a code/comment mismatch in `src/sim_conveyor.cpp`.

19.4 Machine-to-Belt and Agent Logistics

Resource injection is specialized rather than represented by a generic action:

- FoodMachine keeps up to 40% of produced food in the worker's inventory, then tries Storage and compatible conveyors within the implementation's square radius 3.
- MaterialsMachine first deposits construction material onto a visible compatible belt whose directed chain reaches a built OutputMachine. Any remainder enters the worker's inventory for delivery.
- OutputMachine first deposits output onto a compatible nearby belt, then into worker inventory, then nearby Storage, and finally its own machine buffer.
- An agent carrying no output can passively pick up trapped output from an OutputMachine in its eight-neighborhood, subject to remaining total inventory capacity. It deposits only when standing on Storage within Manhattan radius 3 of an Exit.

These radius-3 helper scans include diagonal positions and should not be described as strict adjacency. Agent inventories can mix resource fields up to total capacity 10; the one-resource restriction belongs to each conveyor segment.

19.5 Exit Storage Is the Sole Shipping Drain

`system_ship_out_food()` is a legacy name for output shipping. Each tick it scans Storage at Manhattan radii 1, 2, and 3 around every Exit and removes at most the current output quota. Neither output buffered in a machine nor output in arbitrary Storage is shipped. A conveyor targeting Exit itself also deposits into qualifying Storage first; Exit does not consume the belt directly.

Thus the institutional signal is based on physically shipped output:

$$\text{fill}_t = \begin{cases} \text{shipped}_t/\text{quota}_t, & \text{quota}_t > 0, \\ 1, & \text{otherwise.} \end{cases}$$

Under canonical `external.supply_variant = 1`, an EMA of this fill controls later resource regeneration. Stock trapped outside Exit Storage has no institutional value. `test_output_haul_requires_storage_arrival()` and `test_external_supply_causality()` verify the physical arrival, Manhattan radius, blocked shipping, and next-tick support timing (`tests/simulation_tests.cpp`).

19.6 Degradation, Repair, and Maintenance Priority

Every built belt loses the configured $\delta = 0.0005$ condition during its transport pass, even when empty. Below 0.2 it stops moving resources but remains walkable and repairable. MAINTAIN restores up to `maintain_rate = 0.08` before the same tick's conveyor decay and transport. It consumes time but no material.

Maintenance utility is based on observed degradation, continuous compliance, purpose, hunger suppression, social pressure, and mood. A high-priority belt multiplies this utility by 1.75. Target selection considers degraded belts within the 12-tile observation radius and chooses high priority before distance; execution chooses high priority before worst condition among adjacent belts (`src/sim_utility.cpp`, `src/sim_targets.cpp`, `src/sim_execute.cpp`).

The Director's priority is therefore a visible anonymous signal, not an order. It does not repair the belt, assign an agent, or make an otherwise infeasible action execute (`src/director.h`, `src/sim_director.cpp`).

19.7 Conveyor Construction and the Active Cost Discrepancy

Residents extend routes one frame at a time using a BFS-based site query that prioritizes the OutputMachine-to-Exit path. Frames consume raw material as BUILD progress and auto-select a direction along the proposed route (`Grid::find_conveyor_build_site()`). The construction planner limits pending and total conveyor counts, but it does not optimize a global flow network.

There is currently **no single effective conveyor build cost**:

Route	Stored `build_cost`	Reference
Resident-created frame on Floor	0.5, hard-coded at the call site	`src/sim_execute.cpp`
Four inherited built belts	0.15 from WFC placement	`src/wfc_generator.h`
Director-placed completed belt	1.5, hard-coded	`src/sim_director.cpp`
Generic `Grid::place_new_conveyor` default	1.5	`src/grid.h`

Dismantle refunds are calculated from each tile's stored cost, so these construction paths also produce different refunds. The former unused `conveyor.build_cost` TOML key was removed rather than presented as a universal runtime cost.

19.8 Implemented Mechanism Versus Logistics Hypotheses

The executable establishes physical buffering, type compatibility, capacity, decay, autonomous movement, manual fallback hauling, and prioritized autonomous maintenance. It does not yet establish that agents discover globally efficient, redundant, or robust networks. Single-point failures and collective maintenance failure are plausible consequences, but require metrics and counterfactual tests. The only agent-driven removal policy is dead-end dismantling, documented with its limits in section 20.

19.9 Implementation and Verification References

- State, walkability, route queries, and construction: `src/components.h`, `src/grid.h`, `src/pathfinding.h`.
- Production and resource injection: `src/sim_execute.cpp`, `src/recipes.h`, `src/production.h`.
- Belt transport and shipping: `src/sim_conveyor.cpp`, `src/simulation.cpp`.
- Active defaults: `[production]`, `[conveyor]`, and `[external]` in `config/default.toml`; loading in `src/config.cpp`.
- Map, one-resource, partial-deposit, shipping, and maintenance tests: `tests/simulation_tests.cpp`; metrics contract: `tests/verify_metrics.cmake`.

20 Adaptive Logistics: Implemented Dismantling and Open Hypotheses

The current adaptive-logistics mechanism is narrower than a general dismantle-and-rebuild system. Agents can identify and remove certain dead-end conveyors. They cannot detect conveyor path blockers, are not committed to rebuild what they remove, and do not compare the efficiency of the old and new network. Claims of self-organizing infrastructure therefore remain hypotheses.

20.1 Implemented Candidate: Protected Dead Ends

`Grid::is_dismantle_candidate()` accepts only a built Conveyor whose directed target remains inside the grid and is not:

- Storage or Exit;
- another built Conveyor;
- a built Machine.

A belt pointing outside the grid is rejected rather than classified as a dead end.

It then protects the belt if any cardinal neighbor is a built Machine or built Conveyor. This grace rule prevents the agent from removing a segment adjacent to a chain that may still be under construction. `find_nearest_dead_end_conveyor()` returns the nearest remaining candidate (`src/grid.h`).

Earlier designs also proposed a **path-blocker** category. It is inactive: conveyors are walkable, and `Grid::is_conveyor_blocking_path()` unconditionally returns false. The blocker scans left in utility, target selection, and execution therefore produce no candidate and no priority. DISMANTLE currently means dead-end removal only.

20.2 Selection, Targeting, and Execution

Utility observes candidates within the same 12-tile local window used by agent decision-making. Because the blocker branch is false, the active drive is

$$U_D = c_{\text{eff}}\,(0.6)\,\max(0, 1 - 3u_h)\,\max(0, 2m - 0.5)\,(1 - 0.5\ell)\,g_m,$$

where c_{eff} is effective compliance, u_h is hunger urgency, m is mood, ℓ is laziness, and g_m is the general mood factor. Contrary to the older design, hunger suppresses dismantling rather than boosting it. If the agent already carries more than 2 units of raw material, utility is multiplied by 0.3 (`src/sim_utility.cpp`).

Target selection chooses the nearest dead end within observation radius and sends the agent to a walkable neighboring tile. Execution then scans the agent's eight-neighborhood, rechecks built state and chain protection, and removes the

highest-scoring candidate. With no active blocker score, a valid dead end receives score 1 (`src/sim_targets.cpp`, `src/sim_execute.cpp`).

20.3 Physical Effect and Refund

Successful execution:

1. records the belt's resource type and amount as lost;
2. changes the tile type to Floor;
3. resets built state, progress, condition, and contents;
4. returns build_cost × dismantle_return as raw material, capped by a hard-coded inventory value of 10;
5. stores `dismantled_by`, `dismantled_at_tick`, and `original_type` on the Floor tile;
6. provides a small purpose-need reduction and emits a factual DISMANTLED event.

The default return fraction is 0.5 (`[dismantle]` in `config/default.toml`). The refund uses the tile's stored `build_cost`, not a universal recipe. section 19 documents the unresolved cost split: resident-created, inherited, and Director-created belts currently store different costs, so they yield different nominal refunds.

20.4 There Is No Enforced Rebuild Cycle

After dismantling, the acting agent receives no destination, obligation, sticky state, or reserved material for reconstruction. Ordinary BUILD may later place a frame selected by the independent machine-to-destination BFS planner, but that planner does not know who dismantled the old belt, does not promise to fill the same tile, and does not compare route throughput before and after removal.

Consequently, the executable does not implement the proposed sequence "detect, dismantle, navigate, rebuild at a better position" as one policy. It implements two independent possibilities: removal of a protected dead end and ordinary future construction. Any observed rebuild association must be measured, not inferred from those two mechanisms.

20.5 Repeated Unrebuilt-Site Penalty

`system_check_dismantle_penalties()` scans Floor tiles retaining dismantle metadata after social relationship decay. For default rebuild window $w = 200$, it acts on every tick satisfying

$$w \leq t - t_D \leq w + 50.$$

This is an inclusive 51-tick exposure period, not a one-time penalty. It applies only while the original dismantler is alive. On each exposure tick, every other

living agent within Manhattan distance $d \leq 6$:

- records a negative observation on its own directed edge `observer -> dismantler`, with severity $0.05(1 - d/7)$;
- receives a stress increment of 0.003, clamped to $[0, 1]$.

Thus repeated observers repeatedly lose trust and repeatedly gain stress. The relationship update is disabled when `social_learning_enabled = false`, but the stress increment still occurs because it is applied separately. After $t - t_D > w + 50$, metadata remains but the system performs no further penalty (`src/simulation.cpp`, `src/social.h`).

The scan requires the site still to be Floor. Building any non-Floor structure on that exact tile stops future checks because the tile no longer matches, even though the resident conveyor-build path does not explicitly clear the metadata fields. Director replacement assigns fresh `TileData`, but the penalty scan still depends first on tile type. Rebuilding somewhere else does not clear the original site's exposure. Stopping future checks also does not restore trust or stress already changed.

20.6 What the Mechanism Can and Cannot Support

Claim	Current status
Agents can remove isolated built belts that point nowhere useful	Implemented
Dismantling loses carried belt contents and refunds part of stored cost	Implemented
Nearby observers can repeatedly penalize a surviving dismantler when the exact Floor site remains empty	Implemented
Conveyors can block paths and be removed to reopen corridors	False under current walkability semantics
The dismantler is required or incentivized by explicit state to rebuild	Not implemented
A new segment is guaranteed at the old site or at a better site	Not implemented
The system evaluates route efficiency, redundancy, throughput, or bottlenecks before removal	Not implemented
Repeated dismantlers become socially isolated	Plausible but unverified outcome
Skilled logistics maintainers become valued leaders	Plausible but unverified outcome

Claim	Current status
The network self-organizes toward greater efficiency	Plausible but unverified outcome

No schema-3 field currently links dismantle events, later builds, route length, throughput improvement, or the dismantler's social trajectory. The deterministic suite covers conveyor planning purity, one-resource transport, physical shipping, and maintenance, but it has no focused behavioral test for dead-end dismantle selection or the 51 repeated penalty ticks. These are evidence gaps, not evidence that the proposed outcomes occur.

20.7 Required Evidence for Stronger Claims

A self-organizing-logistics claim would require, at minimum, event linkage between removal and construction, a physical route-quality measure fixed before the experiment, and a multiseed comparison against dismantling disabled. A social-cost claim would additionally require directed trust trajectories with the penalty on and off. Until such tests exist, the academically defensible conclusion is only that dead-end removal and a repeated local social/stress penalty are active.

20.8 Implementation and Verification References

- Candidate, walkability, refund base, and construction planner: `src/grid.h`.
- Utility, targeting, and action effect: `src/sim_utility.cpp`, `src/sim_targets.cpp`, `src/sim_execute.cpp`.
- Repeated penalty and tick placement: `src/simulation.cpp`.
- Directed negative evidence: `src/social.h`.
- Configuration: `[dismantle]` and `[conveyor]` in `config/default.toml`, loaded by `src/config.cpp` into `src/config.h`.
- Existing adjacent logistics coverage and the current focused-test gap: `tests/simulation_tests.cpp`; structured output contract: `tests/verify_metrics.cmake`.

Adams, Tarn. 2014. "Simulation Principles from Dwarf Fortress." In *Game AI Pro 2*. http://www.gameaipro.com/GameAIPro2/GameAIPro2_Chapter41_Simulation_Principles_from_Dwarf_Fortress.pdf.

Burks, Arthur W. 1970. "Essays on Cellular Automata." In *University of Illinois Press*.

Chan, Bert Wang-Chak. 2019. "Lenia — Biology of Artificial Life." In *Complex Systems*, vol. 28. no. 3.

Davis, Q. Tyrell. 2022. *Intention to Explore the Role of Discretization in the Emergence of Self-Organization in Certain Approximations of Continuous Cellular Automata and Other Complex Dynamic Systems.*

Epstein, Joshua M., and Robert Axtell. 1996. *Growing Artificial Societies: Social Science from the Bottom Up.* MIT Press / Brookings Institution Press.

Grimm, Volker, Steven F. Railsback, Cédric E. Vincenot, et al. 2020. "The ODD Protocol for Describing Agent-Based and Other Simulation Models: A Second Update to Improve Clarity, Replication, and Structural Realism." In *Journal of Artificial Societies and Social Simulation*, vol. 23. no. 2. https://www.jasss.org/23/2/7.html.

Gumin, Maxim. 2017. *WaveFunctionCollapse.* https://github.com/mxgmn/WaveFunctionCollapse.

Harabor, Daniel Damir, and Alban Grastien. 2012. "The JPS Pathfinding System." In *Proceedings of the 5th Annual Symposium on Combinatorial Search (SOCS).* https://ojs.aaai.org/index.php/SOCS/article/view/18254.

Hart, Peter E., Nils J. Nilsson, and Bertram Raphael. 1968. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." In *IEEE Transactions on Systems Science and Cybernetics*, vol. 4. no. 2. https://doi.org/10.1109/TSSC.1968.300136.

Karth, Isaac, and Adam Smith. 2017. "WaveFunctionCollapse Is Constraint Solving in the Wild." In *Proceedings of the 12th International Conference on the Foundations of Digital Games (FDG).* https://doi.org/10.1145/3102071.3102087.

Langton, Chris G. 1990. "Computation at the Edge of Chaos: Phase Transitions and Emergent Computation." In *Physica D: Nonlinear Phenomena*, vol. 42. nos. 1-3. https://doi.org/10.1016/0167-2789(90)90064-V.

Lindenmayer, Aristid. 1968. "Mathematical Models for Cellular Interactions in Development." In *Journal of Theoretical Biology*, vol. 18. no. 3. https://doi.org/10.1016/0022-5193(68)90079-9.

Mohri, Mehryar. 2002. "Semiring Frameworks and Algorithms for Shortest-Distance Problems." In *Journal of Automata, Languages and Combinatorics*, vol. 7. no. 4.

Noel, Vincent, Dima Grigoriev, Sergei Vakulenko, and Ovidiu Radulescu. 2013. "Tropicalization and Tropical Equilibration of Chemical Reaction Networks." In *arXiv Preprint arXiv:1303.3963.*

Nowak, Martin A., and Robert M. May. 1992. "Evolutionary Games and Spatial Chaos." In *Nature*, vol. 359. https://doi.org/10.1038/359826a0.

Nystrom, Robert. 2014. *Game Programming Patterns.* Genever Benning. https://gameprogrammingpatterns.com/.

Rafler, Stephan. 2011. *Generalization of Conway's "Game of Life" to a Continuous Domain — SmoothLife.*

Reynolds, Craig W. 1987. "Flocks, Herds and Schools: A Distributed Behavioral Model." In *ACM SIGGRAPH Computer Graphics*, vol. 21. no. 4. https://doi.org/10.1145/37402.37406.

Sandler, Mark, Andrey Zhmoginov, Liangcheng Luo, Alexander Mordvintsev, Ettore Randazzo, and Blaise Agüera y Arcas. 2020. *Image Segmentation via Cellular Automata.*

Schelling, Thomas C. 1971. "Dynamic Models of Segregation." In *Journal of Mathematical Sociology*, vol. 1. no. 2. https://doi.org/10.1080/0022250X.1971.9989794.

Turing, Alan M. 1952. "The Chemical Basis of Morphogenesis." In *Philosophical Transactions of the Royal Society of London B*, vol. 237. no. 641. https://doi.org/10.1098/rstb.1952.0012.

Wolfram, Stephen. 1984. "Universality and Complexity in Cellular Automata." In *Physica D: Nonlinear Phenomena*, vol. 10. nos. 1-2. https://doi.org/10.1016/0167-2789(84)90245-8.

Zhang, Liwen, Gregory Naitzat, and Lek-Heng Lim. 2018. "Tropical Geometry of Deep Neural Networks." In *Proceedings of the 35th International Conference on Machine Learning (ICML).*